TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG

KHOA TOÁN – THỐNG KÊ

BÁO CÁO NHẬP MÔN PYTHON

NHÓM 2

GV hướng dẫn: Đoàn Thị Anh Thư

DANH SÁCH THÀNH VIÊN

HỌ VÀ TÊN	MSSV
HUỲNH XUÂN THÁI	C2000217
LÊ THỊ BÍCH PHƯỢNG	C2000213
VÕ NGỌC THANH	C2000220

MỤC LỤC

CHƯƠNG 1: PYTHON CƠ BẢN *4*

1.1 BẮT ĐẦU VỚI PYTHON **4**

1.1.1 THIẾT LẬP MÔI TRƯỜNG LÀM VIỆC CỦA BẠN 4

1.1.2 BA CÁCH ĐỂ CHẠY PYTHON 7

1.2 TÍNH TOÁN TRONG PYTHON **10**

1.3 QUẢN LÍ GÓI **17**

1.3.1 SỬ DỤNG CÁC GÓI QUẢN LÍ 17

1.3.2 CÀI ĐẶT GÓI TỪ NGUỒN 20

1.4 GIỚI THIỆU JYPYTER NOTEBOOK **20**

1.4.1 BẮT ĐẦU VỚI JUPYTER NOTEBOOK 21

1.4.2 TRONG NOTEBOOK 22

1.4.2 LÀM NHƯ THẾ NÀO ĐỂ ĐÓNG SỔ GHI CHÉP? **23**

1.4.3 TẢI SỔ GHI CHÉP JUPYTER XUỐNG MÁY CHỦ **23**

1.5 BIỂU THỨC LOGIC VÀ TOÁN TỬ **23**

1.6 TỔNG KẾT VÀ BÀI TẬP **27**

1.6.1 TỔNG KẾT 27

1.6.2 BÀI TẬP 27

1.6.3GIẢI 28

CHƯƠNG 2: BIẾN VÀ CẤU TRÚC DỮ LIỆU CƠ BẢN *30*

2.1 BIẾN VÀ PHÉP GÁN **30**

2.2 CẤU TRÚC DỮ LIỆU – STRING **32**

2.3 CẤU TRÚC DỮ LIỆU – LIST **37**

2.4 CẤU TRÚC DỮ LIỆU – TUPLE **40**

2.5 CẤU TRÚC DỮ LIỆU – SET **42**

2.6 CẤU TRÚC DỮ LIỆU – DICTIONARY **43**

2.7 GIỚI THIỆU MẢNG NUMPY **46**

2.8 TỔNG KẾT VÀ BÀI TẬP **54**

2.8.1 TỔNG KẾT 54

2.8.2 BÀI TẬP 55

CHƯƠNG 3: HÀM *57*

3.1 HÀM CƠ BẢN **57**

3.1.1NHỮNG HÀM CÓ SẴN TRONG PYTHON 57

3.1.2TẠO HÀM 58

3.2 Biến địa phương và biến toàn cục **67**

3.3 Hàm lồng **69**

3.4 HÀM LAMBDA **71**

3.5 HÀM DƯỚI DẠNG ĐỐI SỐ CHO HÀM 72
3.6 TỔNG KẾT VÀ BÀI TẬP 73
3.6.1 TỔNG KẾT 73
3.6.2 BÀI TẬP 74
CHƯƠNG 4: CÂU LỆNH RẼ NHÁNH 75
4.1 CÂU LỆNH IF-ELSE 75
4.2 PHÉP TOÁN BA NGÔI 80
4.3 TỔNG KẾT VÀ BÀI TẬP 82
4.3.1 TỔNG KẾT 82
4.3.2 BÀI TẬP 82
CHƯƠNG 5: VÒNG LẶP 84
5.1 VÒNG LẶP FOR 84
5.2 VÒNG LẶP WHILE 91
5.3 BAO HÀM 94
5.3.1 Danh sách bao hàm 95
5.3.2 Thư viện bao hàm 96
5.4 TỔNG KẾT VÀ CÁC BÀI TẬP 96
5.4.1 TÓM TẮT 96
5.4.2 BÀI TẬP 96

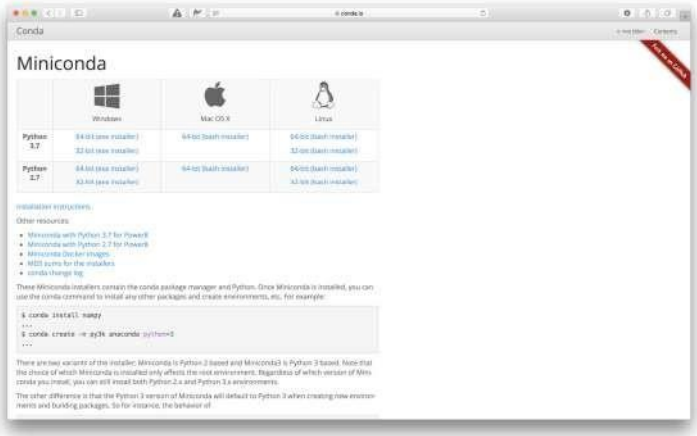
CHƯƠNG 1: PYTHON CƠ BẢN

1.1 BẮT ĐẦU VỚI PYTHON

1.1.1 THIẾT LẬP MÔI TRƯỜNG LÀM VIỆC CỦA BẠN

Bước đầu tiên khi sử dụng Python là thiết lập môi trường làm việc trên máy tính. Phần này giới thiệu các quy trình ban đầu để bắt đầu.

Có nhiều cách khác nhau để cài đặt Python và các gói liên quan, và chúng tôi khuyên bạn nên sử dụng Ana- conda[1] hoặc Miniconda[2] để cài đặt và quản lý các gói của mình. Tùy thuộc vào hệ điều hành (OS) bạn đang sử dụng (tức là Windows, Mac OS X hoặc Linux), bạn sẽ cần tải xuống một trình cài đặt cụ thể cho máy của mình. Cả Anaconda và Miniconda đều nhằm cung cấp các cách dễ dàng để quản lý môi trường làm việc Python trong máy tính khoa học và khoa học dữ liệu.



HÌNH 1.1

Trang tải xuống Miniconda; chọn trình cài đặt dựa trên hệ điều hành của bạn.

Trong ví dụ này, chúng tôi sẽ sử dụng Mac OS X để hướng dẫn bạn cách cài đặt Miniconda (quá trình này rất giống với cài đặt trên Linux). Đối với người dùng Windows, vui lòng bỏ qua phần còn lại của phần này và đọc Phụ lục A về hướng dẫn cài đặt. Sự khác biệt chính giữa Anaconda và Miniconda như sau:

- **Anaconda** là một khung phân phối hoàn chỉnh bao gồm trình thông dịch Python, trình xử lý gói và các gói thường được sử dụng trong máy tính khoa học.
- **Miniconda** là phiên bản "nhẹ" của Anaconda không bao gồm các gói thường được sử dụng. Bạn cần tự cài đặt tất cả các gói khác nhau, nhưng nó bao gồm

trình thông dịch Python và trình quản lý gói.

Tùy chọn chúng tôi đã chọn ở đây là Miniconda và chúng tôi sẽ chỉ cài đặt những gói mà chúng tôi cần. Quá trình cài đặt Miniconda được mô tả dưới đây:

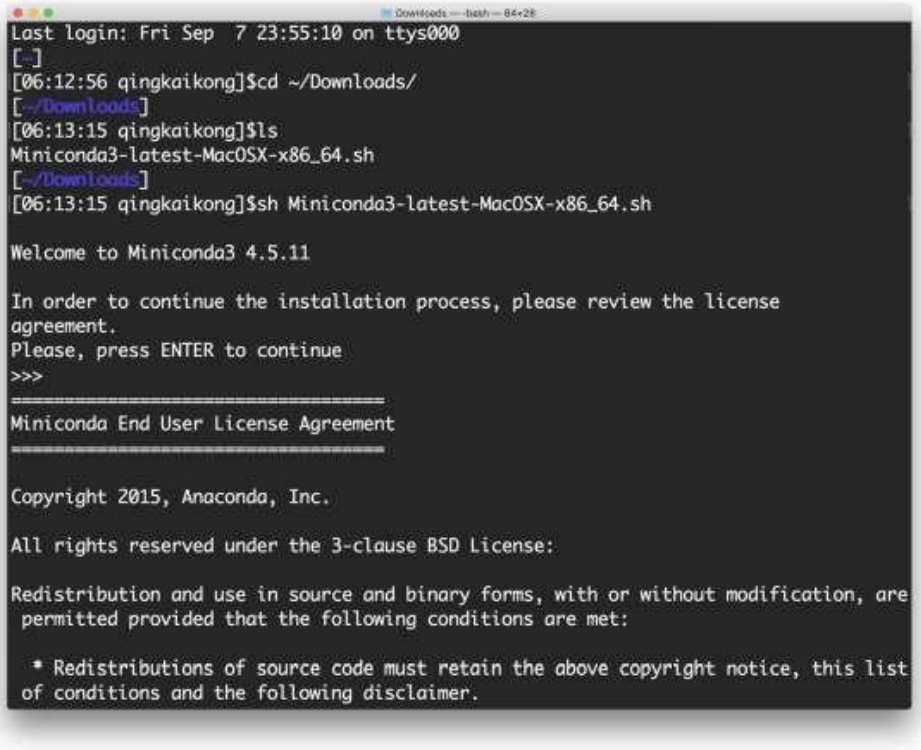
Bước 1. Tải xuống trình cài đặt Miniconda từ trang web.3 Trang tải xuống được hiển thị trong Hình. 1.1. Tại đây, bạn có thể chọn một trình cài đặt khác dựa trên hệ điều hành của mình. Trong ví dụ này, chúng tôi chọn Mac OS X và Python 3.7.

Bước 2. Mở một thiết bị đầu cuối (trên máy Mac, bạn có thể tìm kiếm "thiết bị đầu cuối" trong tìm kiếm Spotlight). Chạy trình cài đặt từ thiết bị đầu cuối bằng các lệnh hiển thị trong Hình. 1.2. Sau khi bạn chạy trình cài đặt, hãy làm theo hướng dẫn để kết thúc quá trình cài đặt.

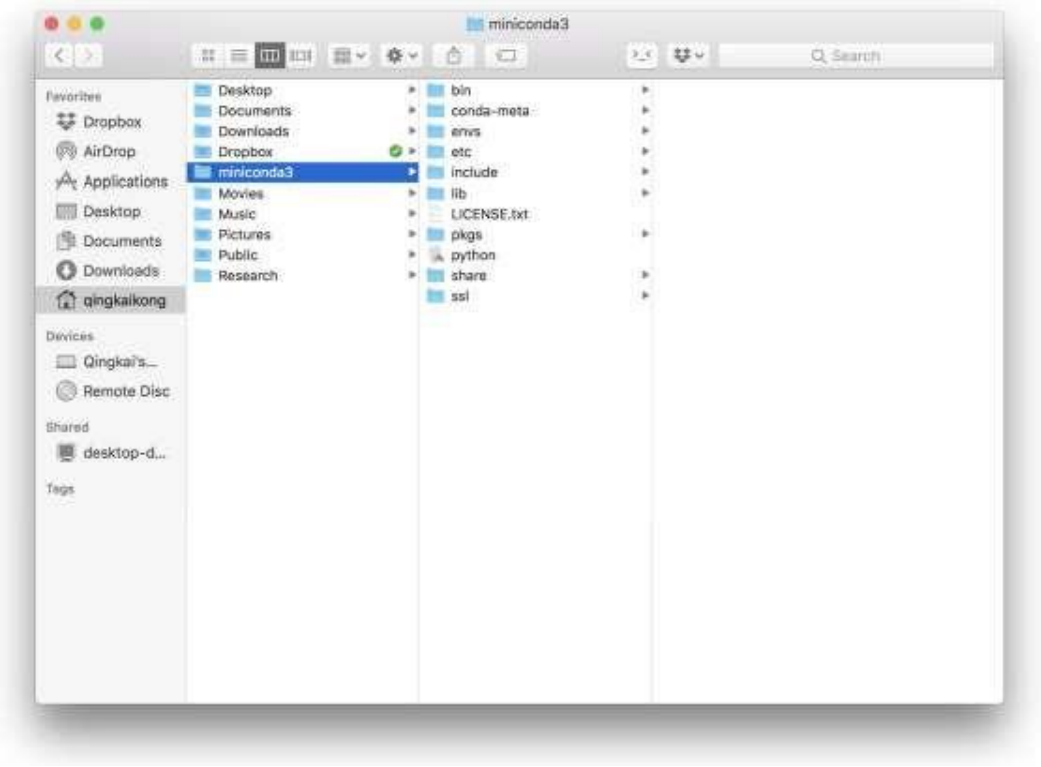
Lưu ý rằng mặc dù bạn có thể thay đổi vị trí cài đặt bằng cách cung cấp cho nó một vị trí thay thế trên máy của bạn, mặc định là thư mục chính của bạn (Hình. 1.3).

Sau khi cài đặt, bạn có thể kiểm tra các gói đã cài đặt bằng cách gõ các lệnh được hiển thị trong Hình. 1.4.

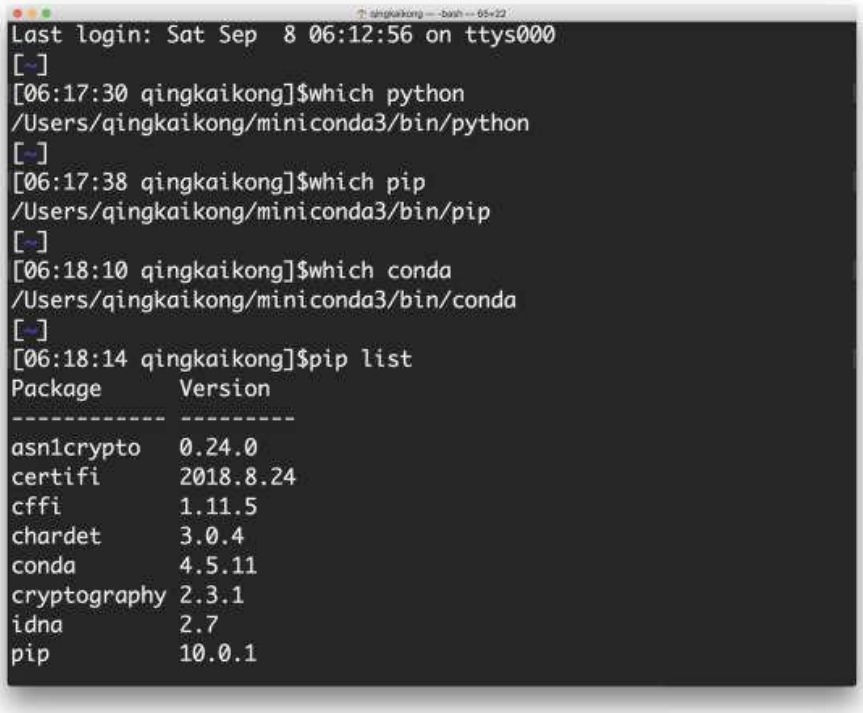
1.1 BẮT ĐẦU VỚI PYTHON



HÌNH 1.2. Ảnh chụp màn hình chạy trình cài đặt trong terminal.

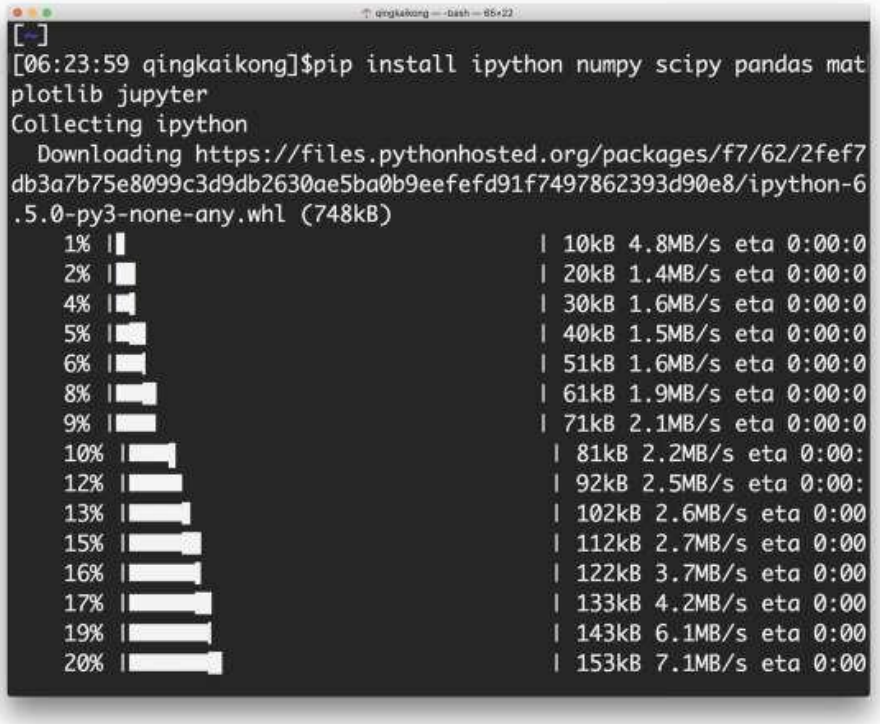


HÌNH 1.3. Vị trí cài đặt mặc định của hệ thống tệp của bạn.



```
Last login: Sat Sep  8 06:12:56 on ttys000
[~]
[06:17:30 qingkaikong]$which python
/Users/qingkaikong/miniconda3/bin/python
[~]
[06:17:38 qingkaikong]$which pip
/Users/qingkaikong/miniconda3/bin/pip
[~]
[06:18:10 qingkaikong]$which conda
/Users/qingkaikong/miniconda3/bin/conda
[~]
[06:18:14 qingkaikong]$pip list
Package      Version
------------ ---------
asn1crypto   0.24.0
certifi      2018.8.24
cffi         1.11.5
chardet      3.0.4
conda        4.5.11
cryptography 2.3.1
idna         2.7
pip          10.0.1
```

HÌNH 1.4.Cách nhanh chóng để kiểm tra xem Miniconda đã được cài đặt thành công và các chương trình đã chạy đúng cách chưa.



HÌNH 1.5. Quá trình cài đặt cho các gói sẽ được sử dụng trong phần còn lại của cuốn sách.

Bước 3. Như hình 1.5, cài đặt các gói cơ bản được sử dụng trong cuốn sách này:ipython, numpy, scipy, pandas, matplotlib, và jupyter notebook. Trong phần sau, chúng ta sẽ nói về việc quản lý các gói sử dụng pipvà conda.

1.1.2 BA CÁCH ĐỂ CHẠY PYTHON

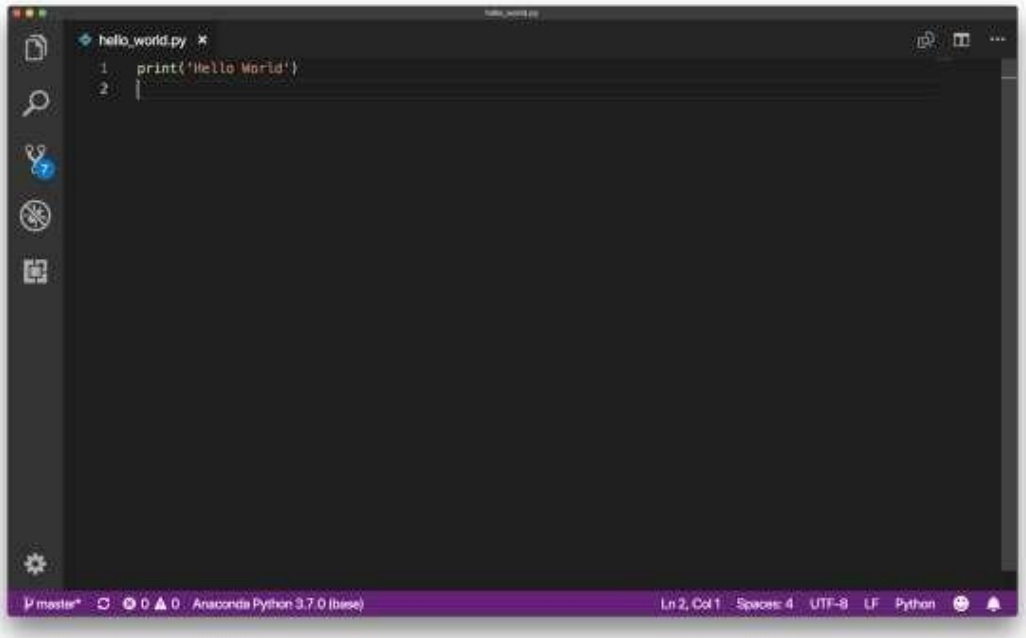
Có nhiều cách khác nhau để chạy mã Python và tất cả chúng đều có cách sử dụng khác nhau. Phần này sẽ giới thiệu ba cách khác nhau để giúp bạn bắt đầu.

Sử dụng cửa sổ dòng lệnh của Python hoặc Ipython(Python Shell hoặc Ipython Shell). Cách dễ nhất để chạy mã Python là thông qua **Python shell** hoặc **IPython shell** (viết tắt của Interactive Python). IPython shell mạnh hơn the Python shell, bao gồm các tính năng như tự động hoàn thành Tab, thông báo lỗi được tô sáng bằng màu, tích hợp UNIX shell cơ bản, v.v. Vì chúng tôi vừa cài đặt IPython, chúng ta hãy thử chạy ví dụ "Hello World" bằng cách sử dụng nó. Để khởi chạy trình bao IPython, chúng ta bắt đầu bằng cách nhập ipython vào terminal(xem Hình 1.6). Sau đó, chúng tôi chạy một lệnh Python bằng cách nhập nó vào cửa sổ dòng lệnh và nhấn Enter. Chúng ta có thể thấy ngay kết quả từ lệnh. Ví dụ: chúng ta có thể in ra "Hello World" bằng cách sử dụng hàm printtrong Python, như trong Hình 1.6.

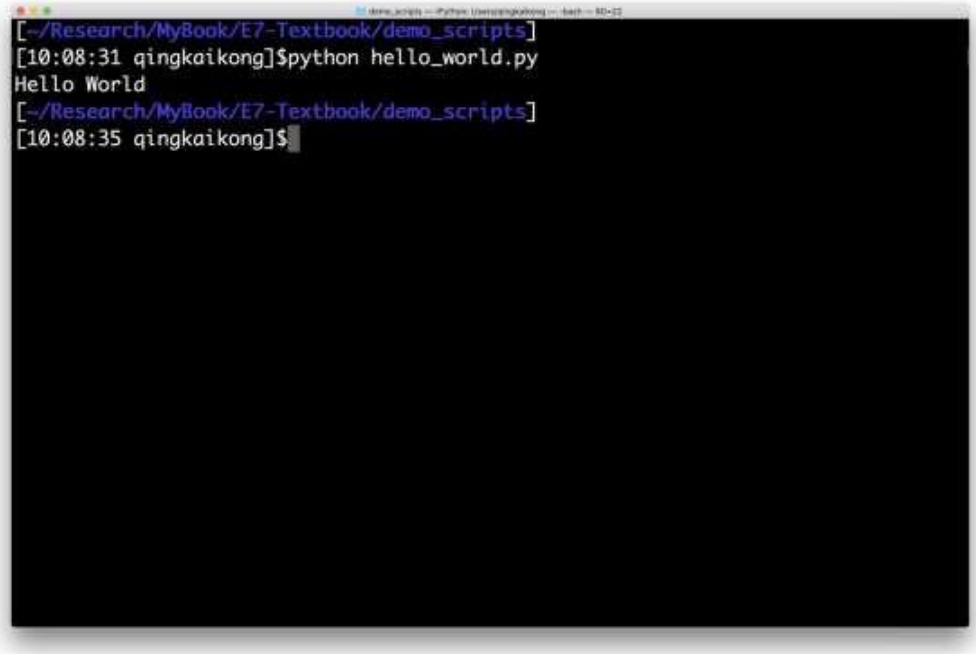
Chạy một tập lệnh / tệp Python từ dòng lệnh. Cách thứ hai để chạy mã Python là

đặt tất cả các lệnh vào một tệp và lưu nó dưới dạng tệp có phần mở rộng.py; phần mở rộng của tệp có thể là bất kỳ thứ gì, nhưng theo quy ước, nó thường là .py. Ví dụ: sử dụng trình soạn thảo văn bản yêu thích của bạn (ở đây chúng tôi đã sử dụng

HÌNH 1.6. Chạy "Hello World" trong IPython shell bằng cách gõ lệnh. "Print" là một hàm sẽ được thảo luận ở phần sau của cuốn sách, nó sẽ in ra bất cứ thứ gì trong dấu ngoặc đơn.

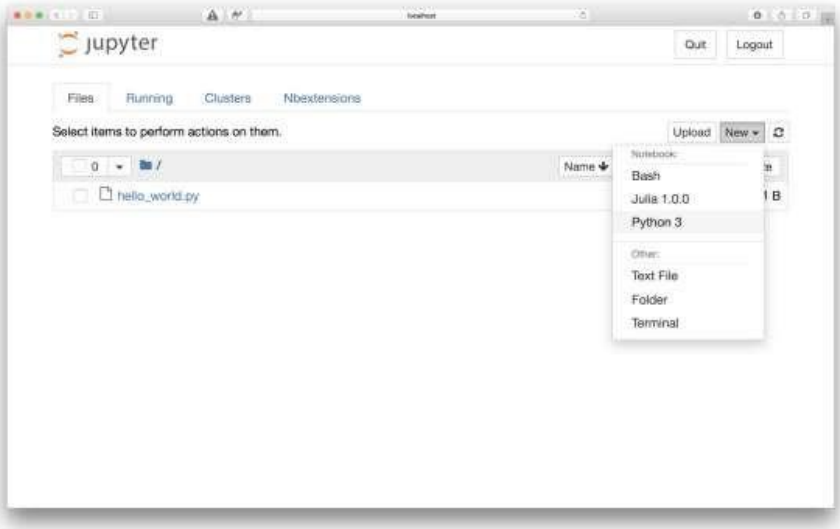


HÌNH 1.7. Ví dụ về ví dụ về tệp tập lệnh Python sử dụng Mã Visual Studio. Nhập các lệnh bạn muốn thực hiện và lưu tệp với tên riêng.



HÌNH 1.8. Để chạy tập lệnh Python từ dòng lệnh, hãy nhập "python hello_world.py". Dòng này yêu cầu Python thực thi các lệnh đã lưu trong tệp này.

Visual Studio Code để nhập các lệnh bạn muốn thực thi trong tệp có tên *hello_world.py*, như được hiển thị trong Hình 1.7, sau đó được chạy từ terminal (xem Hình 1.8).



HÌNH 1.9. Để khởi chạy máy chủ jupyter notebook, nhập jupyter notebook vào dòng lệnh, thao tác này sẽ mở ra một trang trình duyệt như được hiển thị ở đây. Nhấp vào nút "New" ở trên cùng bên phải và chọn "Python3". Thao tác này sẽ tạo một sổ ghi chép Python để chạy mã Python.

S*ử dụng Máy tính xách tay Jupyter*. Cách thứ ba để chạy Python là thông qua **Jupyter Notebook**, đây là một môi trường Python dựa trên trình duyệt rất mạnh mẽ. Chúng ta sẽ thảo luận chi tiết về vấn đề này ở phần sau của chương này. Ví dụ được trình bày ở đây là để chứng minh chúng ta có thể chạy mã nhanh như thế nào bằng cách sử dụng Jupyter notebook. Nếu bạn nhập jupyter notebook vào terminal, một trang web cục bộ sẽ bật lên; sử dụng nút phía trên bên phải để tạo một sổ ghi chép Python3 mới, như thể hiện trong Hình 1.9.

Chạy mã trong Jupyter notebook rất dễ dàng. Nhập mã của bạn vào ô và nhấn Shift + Enter để điều hành tế bào; kết quả sẽ được hiển thị bên dưới mã (Hình 1.10).

1.2 TÍNH TOÁN TRONG PYTHON

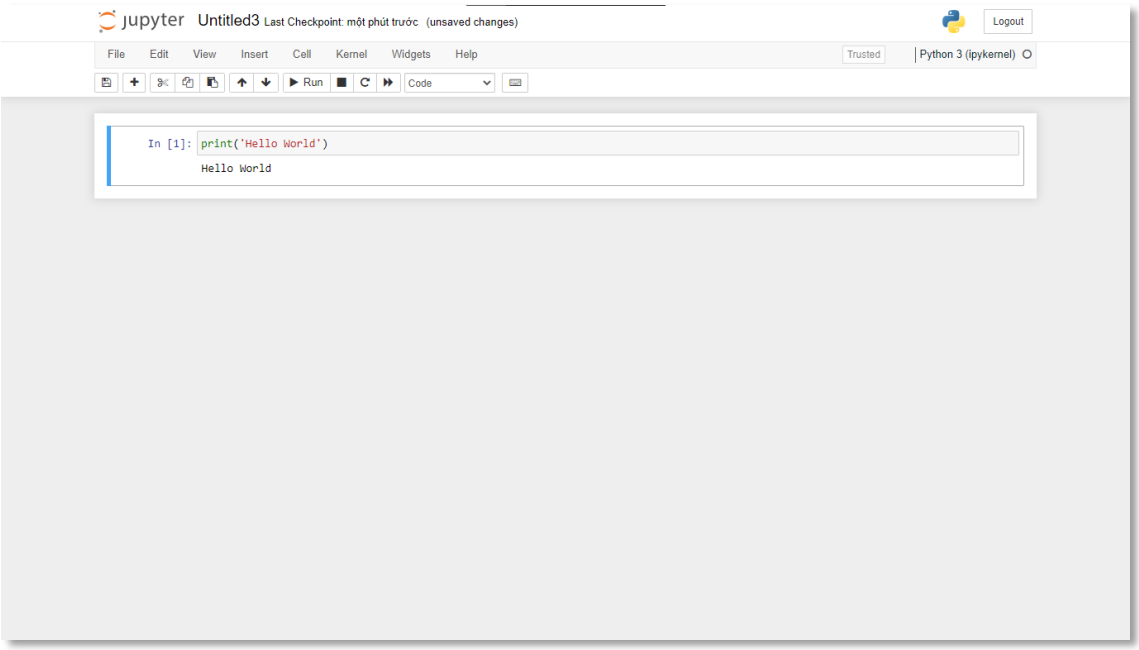
Python chứa các hàm được tìm thấy trong bất kỳ máy tính vẽ đồ thị tiêu chuẩn nào. **Phép toán số học** là phép cộng, trừ, nhân, chia hoặc lũy thừa giữa hai số. **Phép toán hạng số học** là một ký hiệu mà Python đã dành riêng để chỉ một trong các phép toán nói trên. Các ký hiệu này là + cho phép cộng, - cho phép trừ, * cho phép nhân, / cho phép chia và ** cho phép lũy thừa.

Một lệnh hoặc thao tác được **thực thi** khi nó được máy tính giải quyết. Một hướng dẫn được thực thi tại dấu nhắc lệnh bằng cách nhập nó vào nơi bạn thấy biểu tượng

>>> xuất hiện trong cửa sổ dòng lệnh Python (hoặc In [1]: đăng nhập IPython) và sau đó nhấn Enter.

Trong trường hợp Jupyter notebook, hãy nhập thao tác vào ô mã và Shift + Enter. Vì chúng tôi sẽ sử dụng Jupyter notebook cho phần còn lại của sách, để làm quen với tất cả các tùy chọn khác nhau, tất cả các ví dụ trong phần này sẽ được hiển thị trong cửa sổ dòng lệnh IPython - xem phần trước để biết cách bắt đầu làm việc trong IPython.

Đối với người dùng Windows, bạn sẽ sử dụng lời nhắc Anaconda được hiển thị trong Phụ lục A thay vì sử dụng thiết bị đầu cuối.



HÌNH 1.10. Để chạy ví dụ Hello World bên trong Jupyter notebook, hãy nhập lệnh vào 5 ô mã (các hộp màu xám) và nhấn Shift + Enter để thực thi lệnh đó..

THỬ! Tính tổng 1 và 2

In [1]: 1 + 2

Out[1]: 3

Thứ tự các hoạt động là một thứ tự ưu tiên tiêu chuẩn mà các hoạt động khác nhau có mối quan hệ với nhau. Python sử dụng cùng một thứ tự các phép toán mà bạn đã học ở trường phổ thông. Quyền hạn được thực hiện trước phép nhân và phép chia, được thực hiện trước phép cộng và phép trừ. Dấu ngoặc đơn () cũng có thể được sử dụng trong Python để thay thế thứ tự chuẩn của các toán hạn

THỬ! Tính $\frac{3\times4}{(2^2+\frac{4}{2})}$

In [2]: (3*4)/(2**2 + 4/2)
Out[2]: 2.0

TIP! Lưu ý rằng **Out[2]** là giá trị kết quả của thao tác cuối cùng được thực thi. Sử dụng ký hiệu gạch dưới _ đại diện cho kết quả này để chia nhỏ các biểu thức phức tạp thành các lệnh đơn giản hơn.

THỬ! Tính 3 chia cho 4, sau đó nhân kết quả với 2, rồi nâng kết quả lên lũy thừa 3.

```
In [3]: 3/4
Out[3]: 0.75

In [4]: _*2
Out[4]: 1.5

In [5]: _**3
Out[5]: 3.375
```

Python có nhiều hàm số học như sin, cos, tan, asin, acos, atan, exp, log, log10, và sqrtđược lưu trữ trong một mô-đun (sẽ giải thích ở phần sau của chương này) được gọi là **math**. Đầu tiên, nhập mô-đun này để truy cập vào các chức năng này.

```
In [6]: import math
```

TIP! Trong Jupyter notebook và IPython, bạn có thể xem nhanh những gì có trong mô-đun bằng cách nhập module name + dot + TAB. Hơn nữa, nếu bạn nhập một vài ký tự đầu tiên của hàm và nhấn TAB, nó sẽ tự động hoàn thành hàm, được gọi là "Hoàn thành TAB" (ví dụ được hiển thị trong Hình 1.11).

Các hàm toán học này được thực thi thông qua module.function. Các đầu vào cho chúng luôn được đặt bên trong dấu ngoặc đơn được kết nối với tên hàm. Đối với các hàm lượng giác, rất hữu ích khi có giá trị của π. Bạn có thể gọi giá trị này bất kỳ lúc nào bằng cách nhập math.pi vào trình bao IPython.

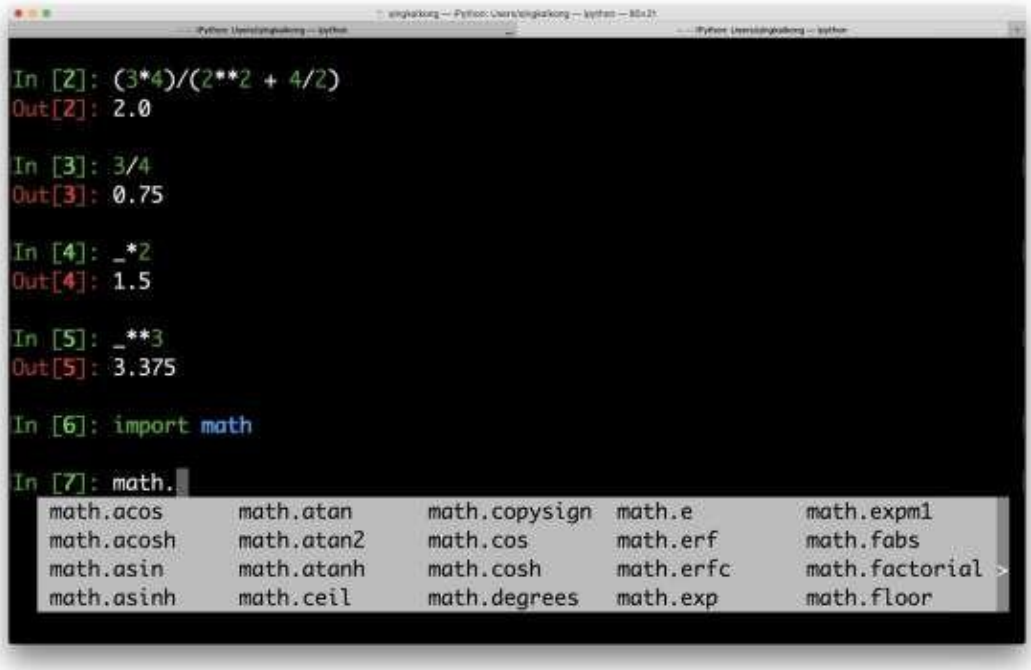
THỬ! Tìm căn bậc hai của 4.

```
In [7]: math.sqrt(4)
Out[7]: 2.0
```

THỬ! Tính $sin(\frac{\pi}{2})$.

```
In [8]: math.sin(math.pi/2)
Out[8]: 1.0
```

Python soạn các hàm như mong đợi, với hàm trong cùng được thực thi đầu tiên. Điều này cũng đúng đối với các lệnh gọi hàm được tạo bằng các phép toán số học.



HÌNH 1.11. Ví dụ minh họa tìm kiếm tương tác cho các chức năng trong IPython bằng cách gõ TAB sau dấu chấm. Hộp màu xám hiển thị tất cả các chức năng có sẵn.

THỬ! Tính $e^{\log 10}$.

In [9]: math.exp(math.log(10))

Out[9]: 10.000000000000002

Lưu ý rằng hàm log trong Python là $\log_e$, hoặc logarit tự nhiên. Nó không phải là $\log_{10}$. Để sử dụng $\log_{10}$, hãy sử dụng hàm math.log10.

TIP! Bạn có thể thấy kết quả ở trên phải là 10, nhưng trong Python, nó hiển thị là 10.000000000000002. Điều này là do tính gần đúng số của Python, được thảo luận trong Chương 9.

Thử ! Tính $e^{\frac{3}{4}}$

```
In [10]: math.exp(3/4)
Out[10]: 2.117000016612675
```

TIP! Sử dụng MŨI TÊN LÊN trong dấu nhắc lệnh sẽ nhớ lại các lệnh đã thực hiện trước đó đã được thực hiện. Nếu bạn vô tình nhập sai một lệnh, bạn có thể sử dụng MŨI TÊN LÊN để gọi lại, sau đó chỉnh sửa nó thay vì gõ lại toàn bộ dòng.

Thông thường, khi sử dụng một hàm trong Python, bạn cần trợ giúp cụ thể cho ngữ cảnh của hàm. Trong IPython or Jupyter notebook, mô tả của bất kỳ chức năng nào có sẵn bằng chức năng gõ function?; dấu chấm hỏi là một lối tắt để được trợ giúp. Nếu bạn thấy một chức năng mà bạn không quen thuộc, cách tốt là sử dụng dấu chấm hỏi trước khi hỏi người hướng dẫn của bạn chức năng cụ thể hoạt động như thế nào.

THỬ! Sử dụng dấu chấm hỏi để tìm định nghĩa của hàm giai thừa.

```
In [11]: math.factorial?
Signature:
math.factorial(x, /)
Docstring:
Find x!.
Raise a ValueError if x is negative or
non-integral. Type:
builtin_function_or_method
```

Python sẽ tăng ZeroDivisionError khi biểu thức 1/0 (là vô cùng) xuất hiện, để nhắc nhở bạn.

TRY IT! 1/0.

```
In [12]: 1/0
.....
ZeroDivisionError   Traceback (most recent call last)
<ipython-input-12-9e1622b385b6> in <module>()
----> 1 1/0
```

Bạn có thể nhập math.inf tại dấu nhắc lệnh để biểu thị vô cực hoặc math.nan để biểu thị một cái gì đó không phải là một số mà bạn muốn được xử lý như một số. Nếu điều này gây nhầm lẫn, sự phân biệt này có thể được bỏ qua ngay bây giờ; nó sẽ được giải thích chi tiết sau. Cuối cùng, Python cũng có thể xử lý các số ảo.

THỬ! Nhập 1/∞, và ∞∗2 để xác minh rằng Python xử lý vô cực như bạn mong đợi.

```
In [13]: 1/math.inf
Out[13]: 0.0

In [14]: math.inf *
2 Out[14]: inf
```

THỬ! TÍNH ∞/∞.

In [15]: math.inf/math.inf
Out[15]: nan

THỬ! Tính tổng 2 + 5i.

In [16]: 2 + 5j

Out[16]: (2+5j)

Lưu ý rằng trong Python, phần ảo được biểu diễn bằng j thay vì i. Một cách khác để biểu diễn một số phức trong Python là sử dụng hàm phức.

```
In [17]: complex(2,5)
Out[17]: (2+5j)
```

Python cũng có thể xử lý ký hiệu khoa học bằng cách sử dụng chữ e giữa hai số. Ví dụ, $1e6$=1000000 và $1e-3$ =0.001.

THỬ! Tính số giây trong 3 năm bằng ký hiệu khoa học.

```
In [18]: 3e0*3.65e2*2.4e1*3.6e3

Out[18]: 94608000.0
```

TIP! Mỗi khi một hàm trong mô-đun math được nhập, nó luôn được nhập là math.function name. Ngoài ra, có một cách đơn giản hơn. Ví dụ, nếu chúng ta muốn sử dụng sin và log từ mô-đun math, chúng ta có thể nhập chúng như sau: từ math nhập sin, log. Với câu lệnh nhập đã sửa đổi này, khi sử dụng các hàm này, hãy sử dụng chúng trực tiếp, ví dụ: sin(20) hoặc log(10)

Các ví dụ trước đã trình bày cách sử dụng Python làm máy tính để xử lý các giá trị dữ liệu khác nhau. Trong Python, có các kiểu dữ liệu bổ sung cần thiết cho các giá trị số: int, float và complex là các kiểu được liên kết với các giá trị này.

- **int**: Số nguyên, chẳng hạn như 1, 2, 3,…
- **float**: Số dấu phẩy động, chẳng hạn như $3.2, 6.4$,…
- **complex**: Số phức, chẳng hạn như $2 + 5j, 3 + 2j$,…

Sử dụng hàm type để kiểm tra kiểu dữ liệu cho các giá trị khác nhau.

THỬ! Tìm ra loại dữ liệu cho 1234.

```
In [19]: type(1234)

Out[19]: int
```

TRY IT! Tìm ra loại dữ liệu cho 3.14.

In [20]: **type**(3.14)

Out[20]: **float**

IPython. Trước khi chuyển sang viết mã phức tạp hơn, chúng ta hãy tiếp tục tìm hiểu thêm về cách quản lý các gói, tức là cách cài đặt, nâng cấp và gỡ bỏ các gói

THỬ! Tìm ra loại dữ liệu cho $2 + 5j$.

In [21]: **type**(2 + 5j)

Out[21]: **complex**

1.3 QUẢN LÍ GÓI

Một tính năng làm cho Python thực sự tuyệt vời là các **gói / mô-đun** khác nhau được phát triển bởi cộng đồng người dùng. Hầu hết thời gian, khi bạn muốn áp dụng một số chức năng hoặc thuật toán, thường bạn sẽ tìm thấy nhiều gói đã có sẵn. Tất cả những gì bạn cần làm là cài đặt các gói và sử dụng chúng trong mã của bạn. Quản lý gói là một trong những kỹ năng quan trọng nhất bạn cần học để tận dụng triệt để lợi thế của Python. Phần này sẽ chỉ cho bạn cách quản lý các gói trong Python.

1.3.1 SỬ DỤNG CÁC GÓI QUẢN LÍ

Ở phần đầu của cuốn sách này, chúng tôi đã cài đặt một số gói sử dụng pip bằng cách nhập pip install pack- age_name. Đây hiện là cách phổ biến và dễ dàng nhất để cài đặt các gói Python. Pip là một trình quản lý gói tự động hóa quá trình cài đặt, cập nhật và gỡ bỏ các gói. Nó có thể cài đặt các gói được xuất bản trên Python Package Index *(PyPI)* .5 Nếu bạn cài đặt Miniconda, pip cũng sẽ có sẵn để bạn sử dụng.

Sử dụng pip help để nhận trợ giúp cho các lệnh khác nhau, như trong Hình 1.12. Các lệnh được sử dụng nhiều nhất thường bao gồm: cài đặt, nâng cấp và gỡ cài đặt gói.

Cài đặt 1 gói

Để cài đặt phiên bản mới nhất của gói:

pip install package_name

Để cài đặt một phiên bản cụ thể, ví dụ: cài đặt phiên bản 1.5:

HÌNH 1.12 Tài liệu trợ giúp của pip sau khi thực thi pip help.

pip install package_name == 1.5

Pip sẽ cài đặt gói cũng như các gói phụ thuộc khác để bạn sử dụng.

Nâng cấp 1 gói

Để nâng cấp gói đã cài đặt lên phiên bản mới nhất từ *PyPI*.

pip install --upgrade package_name

hoặc đơn giản

pip install -U package_name

Gỡ cài đặt 1 gói

pip uninstall package_name

Các lệnh hữu ích khác

Các lệnh hữu ích khác cung cấp thông tin về các gói đã cài đặt có sẵn. Như trong hình 1.13, nếu bạn muốn nhận danh sách tất cả các gói đã cài đặt, bạn có thể sử dụng lệnh:

pip list

Nếu bạn muốn biết thêm về một gói đã cài đặt, chẳng hạn như vị trí của gói, các gói phụ thuộc được yêu cầu khác, v.v., bạn có thể sử dụng lệnh sau như được hiển thị trong Hình 1.14:

pip show package_nam

```
[~]
[14:35:44 qingkaikong]$pip list
Package                              Version
------------------------------- ---------
appnope                              0.1.0
asn1crypto                           0.24.0
astroid                              2.0.4
backcall                             0.1.0
bash-kernel                          0.7.1
bleach                               2.1.4
certifi                              2018.8.24
cffi                                 1.11.5
chardet                              3.0.4
conda                                4.5.11
cryptography                         2.3.1
cycler                               0.10.0
decorator                            4.3.0
defusedxml                           0.5.0
entrypoints                          0.2.3
html5lib                             1.0.1
idna                                 2.7
ipykernel                            4.9.0
ipython                              6.5.0
ipython-genutils                     0.2.0
ipywidgets                           7.4.1
isort                                4.3.4
```

HÌNH 1.13. Sử dụng pip list để hiển thị tất cả các gói được cài đặt trên máy của bạn.

```
[~]
[14:39:19 qingkaikong]$pip show jupyter
Name: jupyter
Version: 1.0.0
Summary: Jupyter metapackage. Install all the Jupyter components in one go.
Home-page: http://jupyter.org
Author: Jupyter Development Team
Author-email: jupyter@googlegroups.org
License: BSD
Location: /Users/qingkaikong/miniconda3/lib/python3.7/site-packages
Requires: jupyter-console, nbconvert, ipykernel, notebook, qtconsole, ipywidgets
Required-by:
[~]
[14:39:27 qingkaikong]$
```

HÌNH 1.14. Sử dụng pip show để nhận thông tin chi tiết về gói đã cài đặt.

Các trình quản lý gói khác tồn tại, ví dụ: conda (được bao gồm trong bản phân phối Anaconda và tương tự như pip về khả năng của nó); do đó, nó không được thảo luận thêm, và bạn có thể tìm thêm thông tin bằng cách đọc tài liệu.

1.3.2 CÀI ĐẶT GÓI TỪ NGUỒN

Đôi khi, bạn sẽ cần tải xuống tệp nguồn cho một số dự án không có trong PyPI. Trong trường hợp đó, bạn cần cài đặt gói theo một cách khác. Trong cài đặt tiêu chuẩn, sau khi giải nén tệp bạn đã tải xuống, thông thường bạn có thể thấy thư mục chứa tập lệnh thiết lập setup.py và tệp có tên README, tài liệu hướng dẫn cách xây dựng và cài đặt mô-đun. Đối với hầu hết các trường hợp, bạn chỉ cần chạy một lệnh từ thiết bị đầu cuối để cài đặt gói:

```
python setup.py install
```

Lưu ý rằng người dùng Windows sẽ cần chạy lệnh sau từ cửa sổ nhắc lệnh:

```
setup.py install
```

Bây giờ bạn đã biết cách quản lý các gói trong PythoN, đây là một bước tiến lớn trong việc sử dụng Python một cách chính xác. Trong phần tiếp theo, chúng tôi sẽ nói thêm về Jupyter notebook mà chúng tôi đã sử dụng cho phần còn lại của cuốn sách.

1.4 GIỚI THIỆU JYPYTER NOTEBOOK

Bạn đã sử dụng cửa sổ dòng lệnh IPython để chạy từng dòng mã. Điều gì sẽ xảy ra nếu bạn có nhiều dòng hơn và muốn chạy nó từng khối và chia sẻ nó một cách dễ dàng với những người khác? Trong trường hợp này, IPython shell không phải là một lựa chọn tốt. Phần này sẽ giới thiệu cho bạn một tùy chọn khác, Jupyter Notebook, mà chúng tôi sẽ sử dụng cho phần còn lại của cuốn sách. Từ Jupyter notebookwebsite[7]:

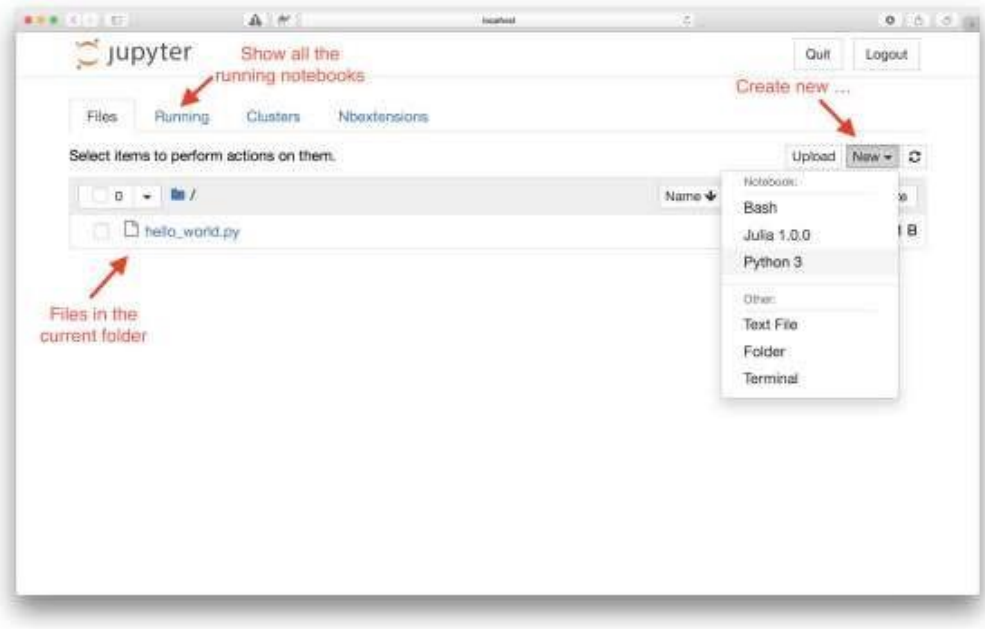
Jupyter notebook là một ứng dụng web mã nguồn mở cho phép bạn tạo và chia sẻ các tài liệu có chứa mã trực tiếp, phương trình, hình ảnh hóa và văn bản tường thuật. Các ứng dụng bao gồm: làm sạch và chuyển đổi dữ liệu, mô phỏng số, lập mô hình thống kê, trực quan hóa dữ liệu, học máy, v.v.

Jupyter notebook chạy bằng trình duyệt của bạn; nó có thể chạy cục bộ trên máy của bạn như một máy chủ cục bộ hoặc từ xa trên máy chủ. Lý do nó được gọi là sổ ghi chép là vì nó có thể chứa mã trực tiếp, các phần tử văn bản đa dạng thức như

phương trình, liên kết, hình ảnh, bảng, v.v. Do đó, bạn có thể có một cuốn sổ ghi chú rất đẹp để mô tả ý tưởng của bạn và mã trực tiếp tất cả trong một tài liệu. Vì vậy, Jupyter notebookđã trở thành một cách phổ biến để kiểm tra ý tưởng, viết blog, báo, và thậm chí là sách. Trên thực tế, cuốn sách này được viết hoàn toàn trong Jupyter notebookvà được chuyển đổi sang LaTeX sau đó. Mặc dù Jupyter notebook có nhiều ưu điểm khác, nhưng để giúp bạn bắt đầu, chúng tôi sẽ chỉ trình bày những điều cơ bản ở đây.

1.4.1 BẮT ĐẦU VỚI JUPYTER NOTEBOOK

Như đã thấy trước đó, chúng ta có thể khởi động sổ ghi chép Jupyter bằng cách nhập lệnh sau vào thiết bị đầu cuối của chúng tôi (đối với người dùng Windows, hãy nhập lệnh này trong dấu nhắc Anaconda) trong thư mục mà bạn muốn định vị sổ ghi chép: jupyter notebook.



HÌNH 1.15. Bảng điều khiển Jupyter notebook sau khi khởi chạy máy chủ. Mũi tên màu đỏ (mũi tên màu xám nhạt trong phiên bản in) đang chỉ cho bạn các tính năng phổ biến nhất trong bảng điều khiển.

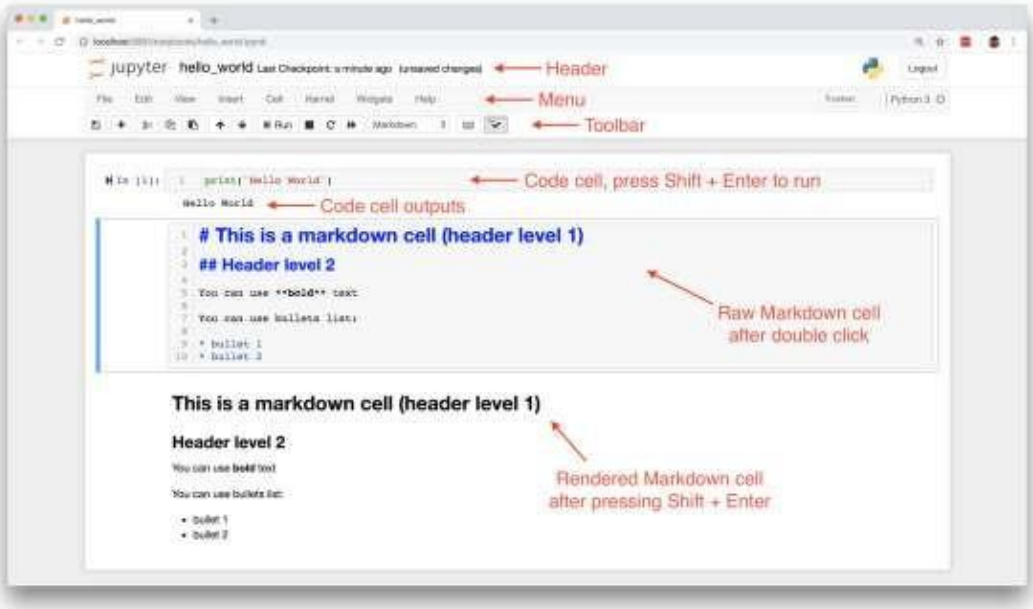
Bảng điều khiển Jupyter notebook sẽ xuất hiện trong trình duyệt như trong Hình 1.15. Địa chỉ mặc định là: http://localhost:8888, tại localhost có cổng 8888, như trong Hình 1.15 (nếu cổng 8888 được sử dụng bởi Jupyter notebooks khác, thì nó sẽ tự động sử dụng cổng khác). Đây thực chất là tạo một máy chủ cục bộ để chạy trong trình duyệt của bạn. Khi bạn điều hướng đến trình duyệt, bạn sẽ thấy một bảng điều khiển. Trong bảng điều khiển này, bạn sẽ thấy một số tính năng quan

trọng được dán nhãn màu đỏ (màu xám nhạt trong phiên bản in). Để tạo một sổ ghi chép Python mới, hãy chọn "Python 3," thường được gọi là hạt nhân Python. Bạn cũng có thể sử dụng Jupyter để chạy các hạt nhân khác. Ví dụ, trong Hình 1.15, có các hạt nhân Bash và Julia mà bạn có thể chạy dưới dạng sổ ghi chép, nhưng bạn sẽ cần phải cài đặt chúng trước. Chúng ta sẽ sử dụng hạt nhân Python, vì vậy hãy chọn hạt nhân Python 3.

1.4.2 TRONG NOTEBOOK

Sau khi bạn tạo một sổ ghi chép Python mới, nó sẽ giống như trong hình 1.16. Thanh công cụ và menu tự giải thích. Nếu bạn di con trỏ qua thanh công cụ, nó sẽ hiển thị cho bạn chức năng của công cụ. Khi bạn nhấn vào menu, nó sẽ hiển thị cho bạn danh sách thả xuống. Một số điều quan trọng cần biết về Jupyter notebook như sau: ô là nơi bạn có thể viết mã hoặc văn bản của mình vào. Nếu bạn chạy ô này, ô này chỉ thực thi mã trong khối ô này. Hai loại ô quan trọng là mã và đánh dấu: (a) ô mã là nơi bạn nhập mã và chạy mã ở đâu; (b) ô đánh dấu là nơi để nhập mô tả ở định dạng văn bản đa dạng thức (xem Hình 1.16 làm ví dụ). Tìm kiếm "Markdown cheatsheet" để bắt đầu nhanh với markdown. Để chạy mã hoặc hiển thị dấu xuống trong sổ tay rất đơn giản: nhấn Shift + Enter.

Sổ ghi chép cho phép bạn di chuyển ô lên hoặc xuống, chèn hoặc xóa ô, v.v. Có rất nhiều tính năng thú vị khác có sẵn bằng sổ ghi chép Jupyter và chúng tôi khuyến khích bạn truy cập nhiều hướng dẫn trực tuyến có sẵn để mở rộng kiến thức của mình.



HÌNH 1.16. Xem nhanh một sổ ghi chép. Đầu trang của sổ ghi chép hiển thị tên của sổ ghi chép. Menu có nhiều danh sách thả xuống khác nhau cho phép bạn truy cập vào tất cả các chức năng của sổ ghi chép. Thanh công cụ cung cấp cho bạn một số phím tắt cho các chức năng thường được sử dụng.

1.4.2 LÀM NHƯ THẾ NÀO ĐỂ ĐÓNG SỔ GHI CHÉP?

Khi bạn đóng tab trình duyệt sổ ghi chép, sổ ghi chép thực sự không được đóng; nó vẫn đang chạy trong nền. Để đóng hoàn toàn sổ ghi chép, hãy đi tới bảng điều khiển và chọn hộp trước sổ ghi chép. Có một tùy chọn tắt máy trong thanh công cụ ở trên; đây là cách chính xác để bạn đóng một cuốn sổ hoàn toàn.

1.4.3 TẢI SỔ GHI CHÉP JUPYTER XUỐNG MÁY CHỦ

Đóng trình duyệt sẽ không tắt máy chủ sổ ghi chép vì nó vẫn đang chạy. Bạn có thể mở lại địa chỉ trước đó trong trình duyệt. Để tắt hoàn toàn, chúng tôi cần đóng thiết bị đầu cuối liên quan mà bạn đã sử dụng để khởi chạy sổ ghi chép.

Bây giờ bạn đã có kiến thức cơ bản để khởi chạy và chạy sổ ghi chép Jupyter, bạn đã được trang bị để tiếp tục học Python.

1.5 BIỂU THỨC LOGIC VÀ TOÁN TỬ

Một biểu thức logic là một câu lệnh có thể đúng hoặc sai. Ví dụ, a <b là một biểu thức logic. Nó có thể đúng hoặc sai tùy thuộc vào giá trị của a và b đã cho. Lưu ý rằng điều này khác với một **biểu thức toán học**, biểu thị một tuyên bố chân lý. Trong ví dụ trước, biểu thức toán học a <b có nghĩa là a nhỏ hơn b và các giá trị của a và b trong đó a b không được phép. Biểu thức logic tạo thành cơ sở của máy tính. Trong

cuốn sách này, tất cả các câu lệnh được giả định là logic hơn là toán học trừ khi có chỉ định khác.

Trong Python, một biểu thức logic đúng sẽ tính thành giá trị True. Một biểu thức sai sẽ tính thành giá trị False. Đây là một kiểu dữ liệu mới được gọi là **boolean**, có các giá trị True và False được tích hợp sẵn. Trong cuốn sách này, "True" tương đương với 1 và "False" tương đương với 0. Biểu thức logic được sử dụng để đặt câu hỏi cho Python. Ví dụ: "3 <4" tương đương với, "3 có nhỏ hơn 4 không?" Vì câu lệnh này đúng, Python sẽ tính nó là 1; tuy nhiên, nếu chúng ta viết 3> 4, điều này là sai và Python sẽ tính nó là 0.

Toán tử so sánh so sánh giá trị của hai số, được sử dụng để xây dựng biểu thức lôgic. Python bảo lưu các ký hiệu>,>, <, <,, để biểu thị "lớn hơn", "lớn hơn hoặc bằng nhau, ""nhỏ hơn ","nhỏ hơn hoặc bằng ","không bằng "và"bằng nhau "tương ứng; xem và Bảng 1.1. Chúng ta hãy bắt đầu với một ví dụ, a = 4, b = 2:

Table 1.1 Comparison operators.

Operator	Description	Example	Results
>	greather than	a > b	True
>=	greater than or equal	a >= b	True
<	less than	a < b	False
<=	less than or equal	a <= b	False
!=	not equal	a != b	True
==	equal	a == b	False

THỬ! Tính toán biểu thức logic cho "5 có bằng 4 không?" và "2 có nhỏ hơn 3 không?"

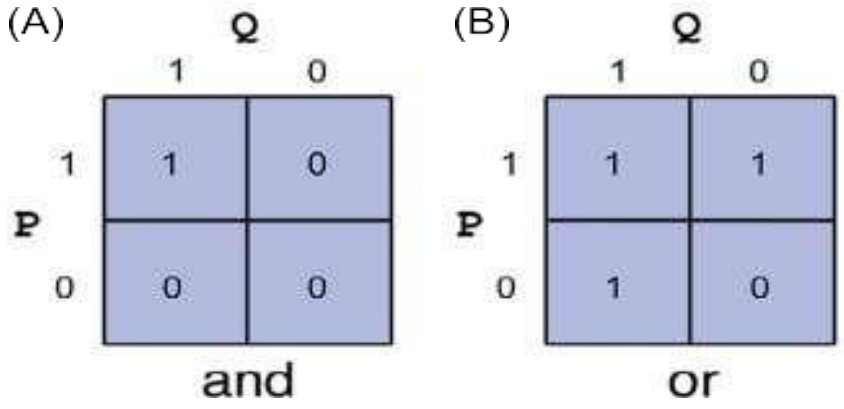
```
In [1]: 5 == 4

Out[1]: False

In [2]: 2<3

Out[2]: True
```

Các toán tử logic, như được trình bày trong Bảng 1.2, là các phép toán giữa hai biểu thức logic mà để tiện cho việc thảo luận, chúng ta sẽ gọi là *P* và *Q*. Các toán tử logic cơ bản mà chúng ta sẽ sử dụng ở đây là **and, or** và **not**.

Table 1.2 Logical operators.

Operator	Description	Example	Results
and	greater than	P and Q	True if both P and Q are True. False otherwise True
or	greater than or equal	P or Q	if either P or Q is True. False otherwise
not	less than	not P	True if P is False. False if P is True

Bảng chân trị, như trong Hình 1.17, của một toán tử hoặc biểu thức logic cho kết quả của mọi tổ hợp chân lý của *P* và *Q*. Hình 1.17 cho thấy các bảng chân lý cho "and" và "or".



HÌNH 1.17. Bảng chân trị cho lôgic and/or.

THỬ! Giả sử *P* là đúng, chúng ta hãy sử dụng Python để xác định xem biểu thức *(P AND NOT (Q)) OR (P AND Q)* luôn đúng bất kể *Q* có đúng hay không. Về mặt logic, bạn có thể thấy tại sao lại như vậy không? Đầu tiên giả sử *Q* là đúng:

```
In [3]: (1 and not 1) or (1 and 1)
Out[3]: 1
```

Now assume *Q* is false

```
In [4]: (1 and not 0) or (1 and
0)
Out[4]: True
```

Cũng giống như với các toán tử số học, các toán tử logic có thứ tự các phép toán tương đối với nhau và liên quan đến các toán tử số học. Tất cả các phép toán số học sẽ được thực hiện trước các phép toán so sánh, các phép toán này sẽ được thực hiện trước các phép toán logic. Dấu ngoặc đơn có thể được sử dụng để thay đổi thứ tự của các hoạt động.

TRY IT! Tính (1 + 3) > (2 + 5)

```
In [5]: 1 + 3 > 2 + 5

Out[5]: False
```

TIP! Ngay cả khi biết thứ tự của các toán tử, thường sẽ giúp cho bạn và các người đọc code của bạn dùng dấu ngoặc đơn dùng để làm rõ hơn ý định của bạn. Trong ví dụ trước (1+3)>(2+5) rõ ràng hơn.

CẢNH BÁO! Trong cách triển khai logic của Python, 1 được sử dụng để biểu thị true và 0 cho false. Nhưng vì 1 và 0 vẫn là số, Python sẽ cho phép các lạm dụng như: (3> 2) + (5> 4), sẽ giải quyết thành 2.

```
In [6]: (3 > 2) + (5 > 4)

Out[6]: 2
```

CẢNH BÁO! Mặc dù trong logic chính thức, 1 được sử dụng để biểu thị đúng và 0 để biểu thị sai, nhưng hệ thống ký hiệu của Python lại khác và nó sẽ nhận bất kỳ số nào không bằng 0 có nghĩa là đúng khi được sử dụng trong một phép toán logic. Ví dụ: 3 và 1 sẽ tính thành true. Không sử dụng tính năng này của Python. Luôn sử dụng 1 để biểu thị một tuyên bố đúng.

TIP! Một hai tuần là khoảng thời gian bao gồm 14 ngày. Sử dụng một biểu thức logic để xác định xem có nhiều hơn 100.000 giây trong một hai tuần hay không.

```
In [7]: (14*24*60*60) > 10000M0

Out[7]: True
```

1.6 TỔNG KẾT VÀ BÀI TẬP

1.6.1 TỔNG KẾT

1. Bây giờ bạn đã học được những kiến thức cơ bản về Python, điều này sẽ cho phép bạn thiết lập môi trường làm việc và thử nghiệm các cách để chạy Python.
2. Python có thể được sử dụng như một máy tính bỏ túi. Nó có tất cả các chức năng và phép toán số học thường được sử dụng với một máy tính khoa học.
3. Bạn có thể quản lý các gói Python bằng cách sử dụng trình quản lý gói.
4. Bạn đã học cách tương tác với sổ ghi chép Jupyter.
5. Bạn cũng có thể sử dụng Python để thực hiện các phép toán logic.
6. Bây giờ bạn đã được giới thiệu về các kiểu dữ liệu int, float, complex, string và boolean trong Python.

1.6.2 BÀI TẬP

7. Tính hệ số góc giữa các điểm (3, 4) và (5, 9). Nhớ lại rằng hệ số góc giữa các điểm (x1, y1) và (x2, y2) là $\frac{y_2-y_1}{x_2-x_1}$.
8. Tính khoảng cách giữa các điểm (3, 4) và (5, 9). Nhớ lại rằng khoảng cách giữa các điểm theo hai chiều là $\sqrt{(x_2-x_1)^2+\ (y_2-y_1)^2}$.
9. Sử dụng hàm giai thừa của Python để tính toán 6!
10. Mặc dù một năm được coi là dài 365 ngày, nhưng một con số chính xác hơn là 365,24 ngày. Kết quả là, nếu chúng ta giữ nguyên 365 ngày tiêu chuẩn trong năm, chúng ta sẽ mất dần phần ngày đó theo thời gian và các mùa cũng như các sự kiện thiên văn khác sẽ không xảy ra như mong đợi. Để giữ cho lịch thời gian luôn đúng, năm nhuận là năm có thêm một ngày, ngày 29 tháng 2, để giữ cho lịch thời gian đi đúng hướng. Năm nhuận xảy ra vào những năm chia hết cho 4, trừ khi nó chính xác chia hết cho 100, trừ khi nó chia hết cho 400. Ví dụ: năm 2004 là năm nhuận, năm 1900 không phải là năm nhuận và năm 2000 là một năm nhuận. Tính số năm nhuận

giữa những năm 1500 và 2010.

11. Một phép gần đúng cho π được phát triển bởi một nhà toán học lỗi lạc tên là Srinivasa Ramanujan. Giá trị gần đúng như sau:

$$\frac{1}{\pi} \approx \frac{2\sqrt{2}}{9801} \sum_{k=0}^{N} \frac{(4k)!\,(1103 + 26390k)}{(k!)^4 396^{4k}}$$

Sử dụng công thức Ramanujan cho N = 0 và N = 1 để tính gần đúng số π. So sánh giá trị gần đúng của bạn với giá trị được lưu trữ của Python cho π. Gợi ý: 0! = 1 theo định nghĩa.

12. Hyperbolic sin hoặc sinh được định nghĩa theo cấp số nhân là sinh (x) = $\frac{\exp(x)-\exp(-x)}{2}$. Tính sinh cho x = 2 bằng cấp số nhân. Xác minh rằng kết quả thực sự là lỗi hypebol bằng cách sử dụng hàm Python sinh trong mô-đun math.

1.6.3 GIẢI

7.

```
In [2]: (x1,y1)=(3,4)
        (x2,y2)=(5,9)
        print((y2-y1)/(x2-x1))

        2.5
```

8.

```
In [30]: import math
         (x1,y1)=(3,4)
         (x2,y2)=(5,9)
         math.sqrt((x2-x1)**2+(y2-y1)**2)

Out[30]: 5.385164807134504
```

9.

```
In [6]: import math
        math.factorial(6)

Out[6]: 720
```

11.

```
In [38]: import math
         kq = 0
         for k in range(0,1):
             kq = kq + ( ( math.factorial(4*k)*(1103 + 26390*k))/((math.factorial(k)**4) * (396 ** (4*k))) )
         kq = ((2*math.sqrt(2))/9801)*kq
         print ("ket qua = ",kq)
         print("pi = ",1/math.pi)

         ket qua =  0.31830987844047015
         pi =  0.3183098861837907
```

12.

```
In [12]: import math
         x = 2
         def hamsin(x):
             a = (math.exp(x) - (1/math.exp(x)))/2
             return a
         print(hamsin(x))
         print(math.sinh(x))
```

```
3.626860407847019
3.6268604078470186
```

CHƯƠNG 2: BIẾN VÀ CẤU TRÚC DỮ LIỆU CƠ BẢN

2.1 BIẾN VÀ PHÉP GÁN

Biến là một chuỗi ký tự và số được liên kết với một phần thông tin. Toán tử gán, được ký hiệu bằng ký hiệu "=", là toán tử được sử dụng để gán giá trị cho các biến trong Python. Dòng x = 1 nhận giá trị đã biết của 1 và gán giá trị đó cho biến có tên "x". Sau khi thực hiện dòng này, số này sẽ được lưu vào biến này. Cho đến khi giá trị bị thay đổi hoặc biến bị xóa, ký tự x sẽ hoạt động giống như giá trị 1.

```
In [1]: x=1
        x

Out[1]: 1
```

Một biến giống như một "vùng chứa" được sử dụng để lưu trữ dữ liệu trong bộ nhớ của máy tính. Tên của biến cho máy tính biết nơi tìm giá trị này trong bộ nhớ. Hiện tại, đủ để biết rằng sổ ghi chép có không gian bộ nhớ riêng để lưu trữ tất cả các biến trong sổ ghi chép. Theo kết quả của ví dụ trước, bạn sẽ thấy các biến x và y trong bộ nhớ. Bạn có thể xem danh sách tất cả các biến trong sổ tay bằng cách sử dụng lệnh % whos

```
In [7]: %whos

        Variable   Type    Data/Info
        ----------------------------
        x          int     1
        y          int     2
```

Ghi chú! Dấu đẳng thức trong lập trình không giống như một câu lệnh đúng trong toán học. Trong toán học, câu lệnh x = 2 khai báo x là 2. Trong lập trình, câu lệnh x = 2 có nghĩa là một giá trị đã biết đang được liên kết với một tên biến, lưu trữ 2 trong x. Mặc dù hoàn toàn hợp lệ khi nói 1 = x trong toán học, nhưng các phép gán trong Python luôn đi sang trái, có nghĩa là giá trị ở bên phải của dấu bằng được gán cho biến ở bên trái của dấu bằng.

Do đó, 1 = x sẽ tạo ra lỗi trong Python. Các toán tử phép gán luôn đứng sau cùng trong thứ tự các phép toán so với các toán tử toán học, logic và so sánh.

Có một số hạn chế đối với các tên biến có thể sử dụng. Các biến chỉ có thể chứa các ký tự chữ và số (chữ cái và số) cũng như dấu gạch dưới. Tuy nhiên, ký tự đầu tiên của tên biến phải là một chữ cái hoặc một dấu gạch dưới. Không cho phép dấu cách trong tên biến và tên biến có phân biệt chữ hoa chữ thường (ví dụ: x và X được coi là các biến khác nhau)

Không giống như trong toán học thuần túy, các biến trong lập trình hầu như luôn đại diện cho một cái gì đó hữu hình. Nó có thể là khoảng cách giữa hai điểm trong không gian hoặc số lượng thỏ trong một quần thể. Do đó, khi code của bạn ngày càng trở nên phức tạp, điều rất quan trọng là các biến của bạn phải mang một cái tên có thể dễ dàng liên kết với những gì chúng đại diện. Ví dụ: khoảng cách giữa hai điểm trong không gian được biểu thị bằng biến dist phù hợp hơn x và số lượng thỏ trong quần thể được biểu thị bằng n_rabbit rõ ràng hơn y.

Lưu ý rằng khi một biến được gán, nó không có bộ nhớ về cách nó được gán. Nghĩa là, nếu giá trị của một biến, y, được xây dựng từ các biến khác, như x, việc gán lại giá trị của x sẽ không thay đổi giá trị của y.

```
In [13]: x=1

 In [2]: y=x+1

In [14]: y
Out[14]: 2

In [15]: x=2

In [16]: y
Out[16]: 2
```

Có thể ghi đè các biến hoặc hàm đã được lưu trữ trong Python. Ví dụ, lệnh help = 2 sẽ lưu giá trị 2 trong biến với tên help. Sau khi gán này, help sẽ hoạt động giống như giá trị 2 thay vì help của hàm. Do đó, phải luôn cẩn thận không đặt các biến trùng tên với các hàm hoặc giá trị tích hợp sẵn.

Điều quan trọng là phải nhớ không bao giờ để lại các lệnh chưa được gán. Một lệnh chưa được gán là một hoạt động có một kết quả, nhưng kết quả đó không được gán cho một biến. Ví dụ, bạn không được sử dụng 2 + 2. Thay vào đó, bạn nên gán nó cho

một số biến x = 2 + 2. Điều này cho phép bạn "giữ" kết quả của các lệnh trước đó giúp cho Python hiểu code dễ hơn.

Bạn có thể xóa một biến khỏi sổ ghi chép bằng cách sử dụng hàm del. Nhập del x sẽ xóa biến x khỏi vùng làm việc. Nếu bạn muốn xóa tất cả các biến trong sổ ghi chép, bạn có thể sử dụng lệnh % reset.

Trong toán học, các biến thường được kết hợp với các số chưa biết; trong lập trình, các biến được liên kết với một giá trị của một kiểu nhất định. Có nhiều kiểu dữ liệu có thể được gán cho các biến.

Kiểu dữ liệu là sự phân loại kiểu thông tin đang được lưu trữ trong một biến. Các kiểu dữ liệu cơ bản mà bạn sẽ sử dụng trong suốt cuốn sách này là boolean, int, float, string, list, tuple, dictionary và set. Mô tả chính thức về các kiểu dữ liệu này được đưa ra trong các phần sau.

2.2 CẤU TRÚC DỮ LIỆU – STRING

Cấu trúc dữ liệu liên quan đến các kiểu mới này là strings, lists, tuples, sets và dictionaries. Chúng ta sẽ bắt đầu với các chuỗi (strings) .

String là một chuỗi các ký tự, chẳng hạn như "Hello World" mà chúng ta đã thấy trong Chương 1. Các chuỗi được bao quanh bởi dấu ngoặc kép đơn hoặc kép. Chúng ta có thể sử dụng hàm print để xuất chuỗi ra màn hình.

THỬ! In ra màn hình câu “I love Python”.

```
In [1]: print("I love Python")

        I love Python
```

Ví dụ: Gán ký tự "S" cho biến có tên là s. Gán chuỗi "Hello World" cho biến w. Xác minh rằng s và w có kiểu chuỗi bằng cách sử dụng hàm type.

```
In [2]: s="S"
        w="Hello World"

In [3]: type(s)
Out[3]: str

In [4]: type(w)
Out[4]: str
```

Ghi chú! Khoảng trống " ", giữa "Hello" và "World" cũng là một kiểu str. Bất kỳ ký hiệu nào cũng có thể là một ký tự, ngay cả những ký tự đã được dành riêng cho các toán tử. Lưu ý rằng là một str, chúng không thực hiện cùng một chức năng. Mặc dù chúng trông giống nhau, nhưng Python diễn giải chúng hoàn toàn khác nhau.

Bởi vì một chuỗi là một mảng các ký tự, nó có độ dài để chỉ ra kích thước của chuỗi. Ví dụ, chúng ta có thể kiểm tra kích thước của chuỗi bằng cách sử dụng hàm len có sẵn.

Character	**H**	**e**	**l**	**l**	**o**		**W**	**o**	**r**	**l**	**d**
Index	0	1	2	3	4	5	6	7	8	9	10

```
In [6]: len(w)
Out[6]: 11
```

Các chuỗi cũng có các chỉ mục cho phép chúng ta tìm vị trí của từng ký tự Chỉ số của vị trí bắt đầu bằng 0.

Chúng ta có thể truy cập một ký tự bằng cách sử dụng một dấu ngoặc và chỉ mục của vị trí. Ví dụ, nếu chúng ta muốn truy cập vào ký tự "W", thì chúng ta nhập như sau:

```
In [7]: w[6]
Out[7]: 'W'
```

[6:11] có nghĩa là vị trí bắt đầu từ chỉ số 6 và vị trí kết thúc là chỉ số 10. Trong phạm vi cắt chuỗi Python, giới hạn trên là độc quyền; điều này có nghĩa là [6:11] sẽ

"cắt" các ký tự từ chỉ mục 6 đến 10. Cú pháp để cắt trong Python là [start: end: step], đối số thứ ba, bước, là tùy chọn. Nếu bạn bỏ qua đối số bước, giá trị mặc định sẽ được đặt thành 1.

Có thể chọn một chuỗi bằng cách sử dụng cắt chuỗi. Ví dụ, nếu chúng ta muốn truy cập "World", chúng ta gõ lệnh sau.

```
In [8]: w[6:11]
Out[8]: 'World'

In [9]: w[6:]
Out[9]: 'World'

In [11]: w[:5]
Out[11]: 'Hello'
```

Có thể sử dụng chỉ số âm khi cắt các chuỗi, có nghĩa là đếm từ cuối chuỗi. Ví dụ, -1 có nghĩa là ký tự cuối cùng, -2 có nghĩa là ký tự thứ hai đến cuối cùng, v.v.

```
In [12]: w[6:-2]
Out[12]: 'Wor'
```

Chuỗi không thể được sử dụng cho các phép toán.

THỬ! Sử dụng "+" để thêm hai số. Xác minh rằng "+" không hoạt động giống như toán tử bổ sung, +.

```
In [14]: 1 "+" 2
  File "C:\Users\Admin\AppData\Local\Temp\ipykernel_11812\425102638.py", line 1
    1 "+" 2
      ^
SyntaxError: invalid syntax
```

Lưu ý! Các số cũng có thể được biểu thị dưới dạng str. Ví dụ: x = '123' có nghĩa là x là chuỗi 123 không phải là số 123. Tuy nhiên, các chuỗi đại diện cho từ hoặc văn bản và do đó không nên có phép cộng được xác định trên chúng.

Một chuỗi có thể được nối với một chuỗi khác. Ví dụ:

```
In [15]: str_a='I love Python!'
         str_b='You too!'
         print(str_a+str_b)

         I love Python!You too!
```

Có thể chuyển đổi các kiểu dữ liệu khác thành chuỗi cũng như sử dụng hàm str tích hợp sẵn. Điều này rất hữu ích, ví dụ, chúng ta có biến x đã lưu trữ 1 dưới dạng kiểu số nguyên, nếu chúng ta muốn in nó ra trực tiếp dưới dạng một chuỗi, chúng ta sẽ gặp lỗi nói rằng chúng ta không thể nối chuỗi với một số nguyên.

```
In [16]: x=1
         print('x= '+x)

         ---------------------------------------------------------------------------
         TypeError                                 Traceback (most recent call last)
         ~\AppData\Local\Temp\ipykernel_11812\3071160287.py in <module>
               1 x=1
         ----> 2 print('x= '+x)

         TypeError: can only concatenate str (not "int") to str
```

Cách chính xác để làm điều đó là chuyển đổi số nguyên thành chuỗi trước, sau đó in nó ra

```
In [17]: print('x = '+str(x))

         x = 1

In [18]: type(str(x))

Out[18]: str
```

Trong Python, chuỗi là một đối tượng có nhiều phương thức khác nhau có thể được sử dụng để thao tác với nó (đây là cái gọi là lập trình hướng đối tượng và sẽ được thảo luận ở phần sau). Để có quyền truy cập vào các phương thức khác nhau này, hãy sử dụng mẫu string.method_name này.

Chuyển biến w thành chữ hoa

```
In [20]: w.upper()

Out[20]: 'HELLO WORLD'
```

Đếm số lần xuất hiện của chữ "l" trong w.

```
In [21]: w.count('l')

Out[21]: 3
```

Thay thế "World" trong biến w thành "Berkeley".

```
In [22]: w.replace("World", "Berkeley")
Out[22]: 'Hello Berkeley'
```

Có nhiều cách khác nhau để định dạng trước một chuỗi. Ở đây chúng tôi giới thiệu hai cách để làm điều đó. Ví dụ: nếu chúng ta có hai biến name và country và chúng ta muốn in chúng ra trong một câu, nhưng chúng ta không muốn sử dụng cách nối chuỗi mà chúng ta đã sử dụng trước đây vì nó sẽ sử dụng nhiều dấu "+" trong chuỗi, chúng ta có thể làm như sau:

```
In [23]: name = "UC Berkeley"
         country = "USA"
         print("%s is a great school in %s!"%(name, country))

         UC Berkeley is a great school in USA!
```

WHAT IS HAPPENING? Trong ví dụ trước,%s trong dấu ngoặc kép cho Python biết rằng chúng ta muốn chèn một số chuỗi tại vị trí này (s là viết tắt của chuỗi trong trường hợp này). % (name, country) là vị trí mà hai chuỗi sẽ được chèn vào.

NEW! Có một cách khác chỉ được giới thiệu trong Python 3.6 trở lên, nó được gọi là f-string, có nghĩa là chuỗi định dạng. Bạn có thể dễ dàng định dạng một chuỗi với dòng sau:

```
In [24]: print(f"{name} is a great school in {country}.")

         UC Berkeley is a great school in USA.
```

Bạn thậm chí có thể in ra một biểu thức số mà không cần chuyển đổi kiểu dữ liệu như chúng ta đã làm trước đây.

THỬ! In kết quả 3 * 4 trực tiếp bằng f-string.

```
In [25]: print(f"{3*4}")

         12
```

Đến đây, chúng ta đã tìm hiểu về cấu trúc dữ liệu chuỗi; đây là cấu trúc dữ liệu trình tự đầu tiên Python.

2.3 CẤU TRÚC DỮ LIỆU – LIST

Trong phần trước, chúng ta đã biết rằng "chuỗi" có thể chứa một chuỗi ký tự. Bây giờ, chúng tôi đang giới thiệu một cấu trúc dữ liệu tuần tự linh hoạt hơn trong Python - list. Cách để xác định nó là sử dụng một cặp dấu ngoặc [ ] và các phần tử bên trong nó được phân tách bằng dấu phẩy. Một danh sách có thể chứa bất kỳ loại dữ liệu nào: số hoặc chuỗi hoặc các loại khác. Ví dụ,

```
In [27]: list_1 = [1, 2, 3]
         list_1

Out[27]: [1, 2, 3]
```

```
In [28]: list_2 = ["Hello", "World"]
         list_2

Out[28]: ['Hello', 'World']
```

Chúng ta cũng có thể đưa các loại hỗn hợp vào list:

```
In [29]: list_3 = [1, 2, 3, "Apple", "orange"]
         list_3

Out[29]: [1, 2, 3, 'Apple', 'orange']
```

Chúng ta cũng có thể lồng các list, ví dụ:

```
In [30]: list_4 = [list_1, list_2]
         list_4

Out[30]: [[1, 2, 3], ['Hello', 'World']]
```

Cách truy xuất phần tử trong list tương tự như cách thực hiện đối với string

List	1	2	3	Apple	Orange
Index	0	1	2	3	4

HÌNH 2.2 Ví dụ về chỉ mục danh sách.

Lấy phần tử thứ 3 trong list_3

```
In [31]: list_3[2]

Out[31]: 3
```

Lấy 3 phần tử đầu tiên trong list_3

```
In [32]: list_3[:3]
Out[32]: [1, 2, 3]
```

Lấy phần tử cuối cùng trong list_3

```
In [33]: list_3[-1]
Out[33]: 'orange'
```

Lấy danh sách đầu tiên từ list_4

```
In [7]: list_4[0]
Out[7]: [1, 2, 3]
```

Tương tự, chúng ta có thể lấy độ dài của danh sách bằng cách sử dụng hàm len.

```
In [8]: len(list_3)
Out[8]: 5
```

Chúng ta cũng có thể nối hai danh sách bằng cách sử dụng dấu +

Gộp list_1 và list_2 vào một list.

```
In [9]: list_1 + list_2
Out[9]: [1, 2, 3, 'Hello', 'World']
```

Các mục mới có thể được thêm vào list hiện có bằng cách sử dụng hàm append từ list.

```
In [10]: list_1.append(4)
         list_1
Out[10]: [1, 2, 3, 4]
```

Ghi chú! Hàm append hoạt động trên chính list như thể hiện trong ví dụ trên, 4 được thêm vào list. Nhưng trong ví dụ list_1 + list_2, list_1 và list_2 sẽ không thay đổi. Bạn có thể kiểm tra list_2 để xác minh điều này.

Chúng ta cũng có thể chèn hoặc xóa phần tử khỏi list bằng cách sử dụng insert và remove, nhưng chúng cũng đang chạy trực tiếp trên list.

```
In [11]: list_1.insert(2,"center")
         list_1

Out[11]: [1, 2, 'center', 3, 4]
```

Ghi chú! Sử dụng remove sẽ chỉ loại bỏ lần xuất hiện đầu tiên của mục. Có một cách khác để xóa một mục bằng cách sử dụng chỉ mục - hàm del.

```
In [12]: del list_1[2]
         list_1

Out[12]: [1, 2, 3, 4]
```

Chúng ta cũng có thể xác định một list trống và thêm vào phần tử mới sau này bằng cách sử dụng append. Nó được sử dụng rất nhiều trong Python khi bạn phải lặp qua một chuỗi các mục; chúng ta sẽ tìm hiểu thêm về cách này trong Chương 5.

Xác định list trống và thêm giá trị 5 và 6 vào list.

```
In [13]: list_5 = []
         list_5.append(5)
         list_5

Out[13]: [5]

In [14]: list_5.append(6)
         list_5

Out[14]: [5, 6]
```

Chúng tôi cũng có thể nhanh chóng kiểm tra xem một phần tử có trong danh sách hay không bằng cách sử dụng toán tử in

Kiểm tra số 5 có trong list_5 không.

```
In [16]: 5 in list_5

Out[16]: True
```

Sử dụng hàm list, chúng ta có thể chuyển các mục trình tự khác thành một list

Chuyển chuỗi "Hello World" thành một list các ký tự.

```
In [17]: list("Hello World")
Out[17]: ['H', 'e', 'l', 'l', 'o', ' ', 'W', 'o', 'r', 'l', 'd']
```

List được sử dụng thường xuyên trong Python khi làm việc với dữ liệu, với nhiều ứng dụng khả thi khác nhau sẽ được thảo luận trong các phần sau.

2.4 CẤU TRÚC DỮ LIỆU – TUPLE

Chúng ta sẽ tìm hiểu thêm một cấu trúc dữ liệu chuỗi khác trong Python - tuple. Nó thường được định nghĩa bằng cách sử dụng một cặp dấu ngoặc đơn () và các phần tử của nó được phân tách bằng dấu phẩy. Ví dụ,

```
In [3]: tuple_1 = (1, 2, 3, 2)
        tuple_1
Out[3]: (1, 2, 3, 2)
```

Với strings và lists, có một cách để lập chỉ mục các bộ giá trị, cắt các phần tử và thậm chí một số cách rất giống với những cách chúng ta đã thấy trước đây.

Lấy length của tuple_1

```
In [4]: len(tuple_1)
Out[4]: 4
```

Lấy các phần tử từ chỉ mục 1 đến 3 cho tuple_1.

```
In [5]: tuple_1[1:4]
Out[5]: (2, 3, 2)
```

Đếm lần xuất hiện của số 2 trong tuple_1

```
In [6]: tuple_1.count(2)
Out[6]: 2
```

Bạn có thể hỏi, sự khác biệt giữa lists và tuples là gì? Nếu chúng tương tự nhau, tại sao chúng ta cần một cấu trúc dữ liệu trình tự khác?

Tuples được tạo ra vì một lý do. Từ tài liệu Python1:

Mặc dù tuples có vẻ tương tự như lists, nhưng chúng thường được sử dụng trong các trường hợp khác nhau và cho các mục đích khác nhau. Các bộ giá trị là bất biến và

thường chứa một chuỗi phần tử không đồng nhất được truy cập thông qua giải nén (xem sau trong phần này) hoặc lập chỉ mục (hoặc thậm chí theo thuộc tính trong trường hợp các bộ giá trị được đặt tên).

Lists có thể thay đổi và các phần tử của chúng thường đồng nhất và được truy cập bằng cách lặp lại list.

```
In [7]: list_1 = [1, 2, 3]
        list_1[2] = 1
        list_1

Out[7]: [1, 2, 1]

In [8]: tuple_1[2] = 1

        ---------------------------------------------------------------------------
        TypeError                                 Traceback (most recent call last)
        ~\AppData\Local\Temp\ipykernel_5436\1592858011.py in <module>
        ----> 1 tuple_1[2] = 1

        TypeError: 'tuple' object does not support item assignment
```

Không đồng nhất có nghĩa là gì? Tuples thường chứa một dãy phần tử không đồng nhất, trong khi Lists thường chứa một dãy đồng nhất. Ví dụ, chúng ta có một list bao gồm các loại trái cây khác nhau. Thông thường, tên của các loại trái cây có thể được lưu trữ trong một list, vì chúng đồng nhất. Bây giờ chúng ta muốn có một cấu trúc dữ liệu để lưu trữ bao nhiêu trái của mỗi loại. Đây thường là lúc dùng đến tuples, vì tên của trái cây và số lượng không đồng nhất. Chẳng hạn như ("apple", 3) có nghĩa là chúng ta có 3 quả táo.

```
In [9]: # a fruit list
        ["apple", "banana", "orange", "pear"]

Out[9]: ['apple', 'banana', 'orange', 'pear']

In [10]: # a list of (fruit, number) pairs
         [("apple",3), ("banana",4),("orange",1), ("pear",4)]

Out[10]: [('apple', 3), ('banana', 4), ('orange', 1), ('pear', 4)]
```

Tuples hoặc Lists có thể được truy cập bằng cách giải nén như thể hiện trong ví dụ sau, yêu cầu số lượng biến ở phía bên trái của dấu đẳng thức bằng số phần tử trong dãy.

```
In [11]: a, b, c = list_1
         print(a, b, c)

         1 2 1
```

Ghi chú! Thao tác ngược lại với việc giải nén là nén, như thể hiện trong ví dụ sau. Chúng ta có thể thấy rằng chúng ta không cần dấu ngoặc đơn để xác định một bộ giá trị, nhưng nó được coi như là một sự lựa chọn tốt để luyện tập

```
In [12]: list_2 = 2, 4, 5
         list_2

Out[12]: (2, 4, 5)
```

2.5 CẤU TRÚC DỮ LIỆU – SET

Một kiểu dữ liệu khác trong Python là Set. Nó là một kiểu có thể lưu trữ một bộ sưu tập không có thứ tự không có phần tử trùng lặp. Nó cũng có thể hỗ trợ các phép toán như hội, giao, hiệu số và hiệu số đối xứng. Nó được xác định bằng cách sử dụng một cặp dấu ngoặc nhọn {} và các phần tử của nó được phân tách bằng dấu phẩy.

```
In [1]: {3, 3, 2, 3, 1, 4, 5, 6, 4, 2}

Out[1]: {1, 2, 3, 4, 5, 6}
```

Sử dụng "Sets" là một cách nhanh chóng để xác định các phần tử duy nhất trong một string, list hoặc tuple.

Tìm các phần tử duy nhất trong list [1, 2, 2, 3, 2, 1, 2]

```
In [2]: set_1 = set([1, 2, 2, 3, 2, 1, 2])
        set_1

Out[2]: {1, 2, 3}
```

Tìm các phần tử duy nhất trong tuple (2, 4, 6, 5, 2).

```
In [3]: set_2 = set((2, 4, 6, 5, 2))
        set_2

Out[3]: {2, 4, 5, 6}
```

Tìm ký tự duy nhất trong string "Banana".

```
In [4]: set("Banana")

Out[4]: {'B', 'a', 'n'}
```

Chúng tôi đã đề cập trước đó rằng các tập hợp hỗ trợ các phép toán như hội, giao, hiệu số và hiệu số đối xứng.

Lấy sự kết hợp của set_1 và set_2.

```
In [5]: print(set_1)
        print(set_2)

        {1, 2, 3}
        {2, 4, 5, 6}

In [6]: set_1.union(set_2)

Out[6]: {1, 2, 3, 4, 5, 6}
```

Lấy giao của set_1 và set_2.

```
In [7]: set_1.intersection(set_2)

Out[7]: {2}
```

set_1 có phải là tập hợp con của {1, 2, 3, 3, 4, 5} không?

```
In [8]: set_1.issubset({1, 2, 3, 3, 4, 5})

Out[8]: True
```

2.6 CẤU TRÚC DỮ LIỆU – DICTIONARY

Chúng ta đã biết đến một số kiểu dữ liệu tuần tự trong các phần trước. Bây giờ chúng ta sẽ học một loại mới và hữu ích - Dictionary, đây là một loại dữ liệu hoàn toàn khác với những loại chúng ta đã đọc trước đó. Thay vì sử dụng một chuỗi số để lập chỉ mục các phần tử (chẳng hạn như lits hoặc tuples), dictionary được lập chỉ mục bằng các key, có thể là một chuỗi, số hoặc tuple (nhưng không phải list). Dictionary bao gồm các key-value và mỗi key ánh xạ tới một value tương ứng. Nó được xác định bằng cách sử dụng một cặp dấu ngoặc nhọn {}, trong khi các phần tử là danh sách các cặp key: value được phân tách bằng dấu phẩy (lưu ý rằng cặp key: value được phân tách bằng dấu hai chấm, với key ở phía trước và value ở cuối) .

```
In [9]: dict_1 = {"apple":3, "orange":4, "pear":2}
        dict_1

Out[9]: {'apple': 3, 'orange': 4, 'pear': 2}
```

Trong dictionary, vì các phần tử được lưu trữ không theo thứ tự, bạn không thể truy cập dictionary dựa trên một chuỗi số chỉ mục. Để truy cập vào một dictionary, chúng ta cần sử dụng key của phần tử - dictionary [key].

THỬ! Lấy nguyên tố "táo" từ dict_1

```
In [10]: dict_1["apple"]

Out[10]: 3
```

Chúng ta có thể lấy key trong dictionary bằng cách sử dụng phương pháp keys hoặc tất cả các values bằng cách sử dụng phương pháp values.

Lấy tất cả keys và values từ dict_1.

```
In [11]: dict_1.keys()

Out[11]: dict_keys(['apple', 'orange', 'pear'])

In [12]: dict_1.values()

Out[12]: dict_values([3, 4, 2])
```

Chúng ta cũng có thể lấy kích thước của dictionary bằng cách sử dụng hàm len.

```
In [13]: len(dict_1)

Out[13]: 3
```

Chúng ta có thể xác định một dictionary trống và sau đó điền phần tử vào sau. Hoặc chúng ta có thể chuyển một list các bộ giá trị có các cặp (key, value) sang một dictionary.

Xác định một dictionary trống có tên school_dict và thêm giá trị "UC Berkeley": "USA".

```
In [14]: school_dict = {}
         school_dict["UC Berkeley"] = "USA"
         school_dict

Out[14]: {'UC Berkeley': 'USA'}
```

Thêm phần tử "Oxford": "UK" vào school_dict.

```
In [15]: school_dict["Oxford"] = "UK"
         school_dict

Out[15]: {'UC Berkeley': 'USA', 'Oxford': 'UK'}
```

Chuyển list các tuples [("UC Berkeley", "USA"), ("Oxford", "UK")] thành một dictionary

```
In [16]: dict([("UC Berkeley", "USA"), ("Oxford", "UK")])

Out[16]: {'UC Berkeley': 'USA', 'Oxford': 'UK'}
```

Chúng ta cũng có thể kiểm tra xem một phần tử có thuộc dictionary hay không bằng cách sử dụng toán tử in.

Xác định xem "UC Berkeley" có thuộc school_dict không

```
In [17]: "UC Berkeley" in school_dict

Out[17]: True
```

Xác định "Harvard" không có trong school_dict.

```
In [18]: "Harvard" not in school_dict

Out[18]: True
```

Chúng ta cũng có thể sử dụng chức năng list để biến một dictionary với một list các key. Ví dụ,

```
In [19]: list(school_dict)

Out[19]: ['UC Berkeley', 'Oxford']
```

2.7 GIỚI THIỆU MẢNG NUMPY

Chương hai của cuốn sách này đã giới thiệu các phương pháp số bằng cách sử dụng Python. Chúng tôi sẽ sử dụng cấu trúc mảng / -matrix rất nhiều trong phần sau của cuốn sách. Để bạn chuẩn bị cho phần này, ở đây chúng tôi sẽ giới thiệu cách phổ biến nhất để xử lý mảng trong Python bằng cách sử dụng mô-đun NumPy. NumPy có lẽ là mô-đun tính toán số cơ bản nhất trong Python.

NumPy được mã hóa bởi cả Python và C. Trên trang web của nó, một số tính năng quan trọng của NumPy được liệt kê như sau:

• Đối tượng mảng N-chiều

• Các chức năng (phát sóng) tinh vi

• Công cụ tích hợp C / C ++ và mã Fortran

• Các khả năng đại số tuyến tính hữu ích, biến đổi Fourier và số ngẫu nhiên

Ở đây, chúng tôi sẽ chỉ giới thiệu cho bạn một phần của mảng NumPy có liên quan đến cấu trúc dữ liệu.

Dần dần, chúng ta sẽ đề cập đến các khía cạnh khác của NumPy trong các chương sau.

Để sử dụng mô-đun NumPy, trước tiên chúng ta cần nhập nó vào. Một cách thông thường để nhập nó là sử dụng np làm tên rút gọn.

```
In [20]: import numpy as np
```

WARNING! Tất nhiên, bạn có thể gọi nó bằng bất kỳ tên nào, nhưng "np" được coi là quy ước và được chấp nhận bởi toàn bộ cộng đồng, và bạn nên sử dụng nó.

Để xác định một mảng trong Python, bạn có thể sử dụng hàm np.array để chuyển đổi danh sách.

Tạo các mảng sau: x=(1 2 3) y=$\begin{pmatrix} 1 & 4 & 3 \\ 9 & 2 & 7 \end{pmatrix}$

```
In [21]: x = np.array([1, 4, 3])
         x

Out[21]: array([1, 4, 3])

In [22]: y = np.array([[1, 4, 3], [9, 2, 7]])
         y

Out[22]: array([[1, 4, 3],
                [9, 2, 7]])
```

Ghi chú! Mảng 2 chiều có thể sử dụng danh sách lồng nhau để biểu diễn, với danh sách bên trong đại diện cho mỗi hàng

Biết kích thước hoặc độ dài của một mảng thường hữu ích. Thuộc tính hình dạng mảng được gọi trên một mảng M và trả về một mảng 2 × 3 trong đó phần tử đầu tiên là số hàng trong ma trận M; và phần tử thứ hai là số cột trong M. Lưu ý rằng đầu ra của thuộc tính shape là một tuple. Thuộc tính size được gọi trên mảng M và trả về tổng số phần tử trong ma trận M.

Tìm các hàng, cột và tổng kích thước cho mảng y.

```
In [23]: y.shape
Out[23]: (2, 3)

In [24]: y.size
Out[24]: 6
```

Ghi chú! Bạn có thể nhận thấy sự khác biệt là chúng tôi chỉ sử dụng y.shape thay vì y.shape (); điều này là do hình dạng là một thuộc tính chứ không phải là một phương thức trong đối tượng mảng này. Chúng tôi sẽ giới thiệu thêm về lập trình hướng đối tượng trong chương sau. Bây giờ, chỉ cần nhớ rằng khi chúng ta gọi một phương thức trong một đối tượng, chúng ta cần sử dụng dấu ngoặc đơn, trong khi với một thuộc tính thì không.

Thông thường, chúng tôi muốn tạo các mảng có cấu trúc hoặc mẫu. Ví dụ, chúng ta có thể tạo mảng z = [1 2 3 ... 2000]. Sẽ rất phức tạp nếu nhập toàn bộ mô tả của z vào Python. Để tạo các mảng có thứ tự và cách đều nhau, sẽ hữu ích khi sử dụng hàm arange trong NumPy.

Tạo một mảng z từ 1 đến 2000 với gia số 1.

```
In [25]: z = np.arange(1, 2000, 1)
         z
Out[25]: array([   1,    2,    3, ..., 1997, 1998, 1999])
```

Sử dụng np.arange, chúng ta có thể tạo z một cách dễ dàng. Hai số đầu tiên là số bắt đầu và kết thúc của dãy số và số cuối cùng là số gia. Vì rất phổ biến có số gia là 1, nếu số gia không được chỉ định, Python sẽ sử dụng giá trị mặc định là 1. Do đó np.arange

(1, 2000) sẽ có cùng kết quả là np.arange (1, 2000) , 1). Cũng có thể sử dụng số tăng âm hoặc không tăng dần. Nếu gia số "bỏ lỡ" giá trị cuối cùng, nó sẽ chỉ kéo dài cho đến giá trị ngay trước giá trị kết thúc.

Ví dụ: x = np.arange (1,8,2) sẽ là [1, 3, 5, 7].

Tạo một mảng với [0.5, 1, 1.5, 2, 2.5].

```
In [26]: np.arange(0.5, 3, 0.5)

Out[26]: array([0.5, 1. , 1.5, 2. , 2.5])
```

Đôi khi chúng ta muốn đảm bảo điểm bắt đầu và điểm kết thúc cho một mảng nhưng vẫn có các phần tử cách đều nhau. Ví dụ, chúng ta có thể muốn một mảng bắt đầu bằng 1, kết thúc bằng 8 và có chính xác 10 phần tử.

Để thực hiện việc này, hãy sử dụng hàm np.linspace. Hàm linspace nhận ba giá trị đầu vào được phân tách bằng dấu phẩy; do đó, A = linspace (a, b, n) tạo ra một mảng n phần tử cách đều nhau bắt đầu từ a và kết thúc tại b.

Sử dụng linspace để tạo một mảng bắt đầu từ 3, kết thúc bằng 9 và chứa 10 phần tử

```
In [27]: np.linspace(3, 9, 10)

Out[27]: array([3.        , 3.66666667, 4.33333333, 5.        , 5.66666667,
                6.33333333, 7.        , 7.66666667, 8.33333333, 9.        ])
```

Truy cập vào mảng 1 chiều NumPy tương tự như những gì chúng tôi đã mô tả cho list hoặc tuple: nó có một chỉ mục để chỉ ra vị trí. Ví dụ,

```
In [28]: # get the 2nd element of x
         x[1]

Out[28]: 4

In [29]: # get all the element after the 2nd element of x
         x[1:]

Out[29]: array([4, 3])

In [30]: # get the last element of x
         x[-1]

Out[30]: 3
```

Đối với mảng 2 chiều, nó hơi khác một chút, vì chúng ta có các hàng và cột. Để có quyền truy cập vào dữ liệu trong mảng 2 chiều M, chúng ta cần sử dụng M [r, c], theo đó hàng r và cột c được phân tách bằng dấu phẩy. Điều này được gọi là "lập chỉ mục mảng." R và c có thể là số đơn, danh sách, v.v. Nếu bạn chỉ nghĩ về chỉ số hàng hoặc chỉ số cột, thì nó tương tự với mảng 1 chiều.

Ví dụ: y= $\begin{pmatrix} 1 & 4 & 3 \\ 9 & 2 & 7 \end{pmatrix}$

Lấy phần tử ở hàng đầu tiên và cột thứ hai của mảng y.

```
In [31]: y[0,1]
Out[31]: 4
```

Lấy hàng đầu tiên của mảng y.

```
In [32]: y[0, :]
Out[32]: array([1, 4, 3])
```

Lấy cột cuối cùng của mảng y.

```
In [33]: y[:, -1]
Out[33]: array([3, 7])
```

Lấy cột đầu tiên và cột thứ ba của mảng y.

```
In [34]: y[:, [0, 2]]
Out[34]: array([[1, 3],
                [9, 7]])
```

Dưới đây là một số mảng được xác định trước thực sự hữu ích: np.zeros, np.ones và np.empty là ba hàm hữu ích. Xem ví dụ về các mảng được xác định trước này bên dưới:

Tạo một mảng 3 × 5 với tất cả các phần tử là 0.

```
In [35]: np.zeros((3, 5))
Out[35]: array([[0., 0., 0., 0., 0.],
                [0., 0., 0., 0., 0.],
                [0., 0., 0., 0., 0.]])
```

Tạo mảng 5 × 3 với tất cả các phần tử là 1

```
In [36]: np.ones((5, 3))

Out[36]: array([[1., 1., 1.],
                [1., 1., 1.],
                [1., 1., 1.],
                [1., 1., 1.],
                [1., 1., 1.]])
```

Ghi chú! Hình dạng của mảng được xác định trong một tuple với số hàng là mục đầu tiên và số cột là mục thứ hai. Nếu bạn chỉ cần một mảng 1 chiều, thì chỉ sử dụng một số làm đầu vào: np.ones (5).

Tạo một mảng trống 1 chiều có 3 phần tử

```
In [37]: np.empty(3)

Out[37]: array([4.24399158e-314, 4.23762559e-311, 4.24186959e-311])
```

Ghi chú! Mảng trống không thực sự trống; nó chứa đầy các số rất nhỏ ngẫu nhiên.

Bạn có thể gán lại giá trị của một mảng bằng cách sử dụng lập chỉ mục mảng và toán tử gán. Bạn có thể gán lại nhiều phần tử cho một số duy nhất bằng cách sử dụng lập chỉ mục mảng ở phía bên trái. Bạn cũng có thể gán lại nhiều phần tử của một mảng miễn là cả số phần tử được gán và số phần tử được gán đều giống nhau. Bạn có thể tạo một mảng bằng cách sử dụng lập chỉ mục mảng.

Cho a = [1, 2, 3, 4, 5, 6]. Gán lại phần tử thứ tư của A thành 7. Gán lại phần tử thứ nhất, thứ hai và thứ ba cho 1. Gán lại phần tử thứ hai, thứ ba và thứ tư cho 9, 8 và 7.

```
In [38]: a = np.arange(1, 7)
         a

Out[38]: array([1, 2, 3, 4, 5, 6])

In [39]: a[3] = 7
         a

Out[39]: array([1, 2, 3, 7, 5, 6])

In [41]: a[:3] = 1
         a

Out[41]: array([1, 1, 1, 7, 5, 6])

In [42]: a[1:4] = [9, 8, 7]
         a

Out[42]: array([1, 9, 8, 7, 5, 6])
```

Tạo một mảng b[2x2] và đặt b= $\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$ sử dụng lập chỉ mục mảng.

```
In [43]: b = np.zeros((2, 2))
         b[0, 0] = 1
         b[0, 1] = 2
         b[1, 0] = 3
         b[1, 1] = 4
         b

Out[43]: array([[1., 2.],
                [3., 4.]])
```

Mảng được xác định bằng cách sử dụng số học cơ bản; tuy nhiên, có các phép toán giữa một đại lượng vô hướng (một số duy nhất) và một mảng và các phép toán giữa hai mảng. Chúng ta sẽ bắt đầu với các phép toán giữa một đại lượng vô hướng và một mảng. Để minh họa, hãy cho c là một đại lượng vô hướng và b là một ma trận. Khi đó b + c, b-c, b * c và b / c cộng a với mọi phần tử của b, trừ c với mọi phần tử của b, nhân mọi phần tử của b với c và chia mọi phần tử của b cho c tương ứng.

Đặt b= $\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$. Cộng và trừ 2 với b. Nhân và chia b với 2. Bình phương mọi phần tử của b. Cho c là một vô hướng. Tự mình kiểm chứng tính phản xạ của phép cộng và phép nhân vô hướng: b + c = c + b và cb = bc.

```
In [24]: b + 2

Out[24]: array([[3., 4.],
                [5., 6.]])

In [25]: b - 2

Out[25]: array([[-1.,  0.],
                [ 1.,  2.]])

In [26]: 2 * b

Out[26]: array([[2., 4.],
                [6., 8.]])

In [27]: b/2

Out[27]: array([[0.5, 1. ],
                [1.5, 2. ]])

In [28]: b**2

Out[28]: array([[ 1.,  4.],
                [ 9., 16.]])
```

Việc mô tả các phép toán giữa hai ma trận phức tạp hơn. Cho b và d là hai ma trận có cùng kích thước. Khi đó b-d lấy mọi phần tử của b và trừ đi phần tử tương ứng của d. Tương tự, b + d thêm mọi phần tử của d vào phần tử tương ứng của b.

Đặt b= $\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$ và d= $\begin{pmatrix} 3 & 4 \\ 5 & 6 \end{pmatrix}$. Tính b+d và b-d

```
In [29]: b = np.array([[1, 2], [3, 4]])
         d = np.array([[3, 4], [5, 6]])

In [30]: b+d

Out[30]: array([[ 4,  6],
                [ 8, 10]])

In [31]: b-d

Out[31]: array([[-2, -2],
                [-2, -2]])
```

Có hai loại nhân (và chia) khác nhau cho ma trận. Có phép nhân ma trận theo từng phần tử và phép nhân ma trận tiêu chuẩn. Phần này sẽ chỉ trình bày cách hoạt động của phép nhân và chia ma trận từng phần tử. Phép nhân ma trận chuẩn sẽ được mô tả trong chương sau về Đại số tuyến tính. Python lấy biểu tượng * để có nghĩa là phép nhân

tăng tốc phần tử. Đối với ma trận b và d có cùng kích thước, b * d lấy mọi phần tử của b và nhân nó với phần tử tương ứng của d. Điều này cũng đúng cho / và **.

Tính b * d, b / d và b ** d.

```
In [35]: b * d

Out[35]: array([[ 3,  8],
                [15, 24]])

In [38]: b/d

Out[38]: array([[0.33333333, 0.5       ],
                [0.6       , 0.66666667]])

In [40]: b**d

Out[40]: array([[   1,   16],
                [ 243, 4096]], dtype=int32)
```

Chuyển vị của một mảng, b, là một mảng, d, trong đó b [i, j] = d [j, i]. Nói cách khác, phép chuyển vị sẽ chuyển các hàng và cột của b. Bạn có thể chuyển một mảng trong Python bằng cách sử dụng phương thức mảng T.

Tính chuyển vị của mảng b.

```
In [41]: b.T

Out[41]: array([[1, 3],
                [2, 4]])
```

NumPy có nhiều hàm số học, chẳng hạn như sin, cos, v.v., có thể lấy mảng làm đối số đầu vào. Đầu ra là hàm được đánh giá cho mọi phần tử của mảng đầu vào. Một hàm nhận một mảng làm đầu vào và thực hiện chức năng trên nó được cho là được vectơ hóa.

Tính np.sqrt cho x = [1, 4, 9, 16]

```
In [42]: x = [1, 4, 9, 16]
         np.sqrt(x)

Out[42]: array([1., 2., 3., 4.])
```

Các phép toán logic chỉ được định nghĩa giữa một đại lượng vô hướng và một mảng và giữa hai mảng có cùng kích thước. Giữa đại lượng vô hướng và một mảng,

phép toán logic được thực hiện giữa đại lượng vô hướng và từng phần tử của mảng. Giữa hai mảng, phép toán logic được tiến hành từng phần tử.

THỬ! Kiểm tra xem phần tử nào của mảng x = [1, 2, 4, 5, 9, 3] lớn hơn 3.

Kiểm tra xem phần tử nào trong x lớn hơn phần tử tương ứng trong y = [0, 2, 3, 1, 2, 3].

```
In [43]: x = np.array([1, 2, 4, 5, 9, 3])
         y = np.array([0, 2, 3, 1, 2, 3])

In [44]: x > 3

Out[44]: array([False, False,  True,  True,  True, False])

In [45]: x > y

Out[45]: array([ True, False,  True,  True,  True, False])
```

Python có thể lập chỉ mục các phần tử của một mảng đáp ứng một biểu thức logic.

Gọi x là mảng giống như trong ví dụ trước. Tạo một biến y chứa tất cả các phần tử của x lớn hơn 3. Gán tất cả các giá trị của x lớn hơn 3, giá trị 0.

```
In [46]: y = x[x > 3]
         y

Out[46]: array([4, 5, 9])

In [47]: x[x > 3] = 0
         x

Out[47]: array([1, 2, 0, 0, 0, 3])
```

2.8 TỔNG KẾT VÀ BÀI TẬP

2.8.1 TỔNG KẾT

1. Lưu trữ, truy xuất và xử lý thông tin và dữ liệu là quan trọng trong bất kỳ lĩnh vực khoa học và kỹ thuật nào.

2. Gán biến là một công cụ quan trọng để xử lý các giá trị dữ liệu.

3. Có các kiểu dữ liệu khác nhau để lưu trữ thông tin trong Python: int, float và boolean cho các giá trị đơn lẻ và strings, lists, tuples, sets và dictionary cho dữ liệu tuần tự.

4. Mảng NumPy là một cấu trúc dữ liệu mạnh được sử dụng rất nhiều trong tính toán khoa học.

2.8.2 BÀI TẬP

3. Cho x = 10 và y = 3. Viết code để thực hiện mỗi nhiệm vụ sau đây.

u=x+y

v = xy

w = x/y

z = sin(x)

r = 8sin(x)

s = 5sin(xy)

p=x**y

```
In [59]: x=10
         y=3

In [50]: u=x+y
         u

Out[50]: 13

In [52]: v=x*y
         v

Out[52]: 30

In [53]: w=x/y
         w

Out[53]: 3.3333333333333335

In [66]: z=np.sin(x)
         z

Out[66]: -0.5440211108893698

In [68]: r=8*np.sin(x)
         r

Out[68]: -4.352168887114958

In [69]: s=5*np.sin(x*y)
         s

Out[69]: -4.940158120464309

In [70]: p=x**y
         p

Out[70]: 1000
```

16. Gán (2, 3, 2) cho set_a và (1, 2, 3) cho set_b. Lấy những thứ sau:

• Hợp của set_a và set_b

• Giao của set_a và set_b

• Sự khác biệt của set_a với set_b bằng cách sử dụng hàm difference.

```
In [71]: set_a=set((2,3,2))
         set_b=set((1,2,3))

In [72]: set_a.union(set_b)
Out[72]: {1, 2, 3}

In [73]: set_a.intersection(set_b)
Out[73]: {2, 3}

In [76]: set_a.difference(set_b)
Out[76]: set()

In [77]: set_b.difference(set_a)
Out[77]: {1}
```

22. Tạo một mảng y = $\begin{pmatrix} 3 & 5 & 3 \\ 2 & 2 & 5 \\ 3 & 8 & 9 \end{pmatrix}$ và tính chuyển vị của nó

```
In [78]: y=np.array([[3,5,3],[2,2,5],[3,8,9]])
         y
Out[78]: array([[3, 5, 3],
                [2, 2, 5],
                [3, 8, 9]])

In [79]: y.T
Out[79]: array([[3, 2, 3],
                [5, 2, 8],
                [3, 5, 9]])
```

CHƯƠNG 3: HÀM

3.1 HÀM CƠ BẢN

Trong chương trình, hàm là một chuỗi tập hợp các chỉ thị để thực hiện một tác vụ cụ thể. Khi gọi một hàm bất kì thì những câu lệnh trong hàm đó sẽ được chạy để thực hiện những thao tác mà người viết chương trình mong muốn. Hàm cũng có thể nhập vào các đối số bởi người dùng (thực thể gọi hàm). Hàm cũng có thể xuất ra các thông số và đó cũng là những kết quả có được sau khi hàm thực hiện xong. Ví dụ như hàm math.sin đối số nhập vào là một góc (radians) và đối số xuất ra là một giá trị gần bằng với sin góc được nhập vào. Chuỗi tập hợp các chỉ dẫn để tính giá trị cần bằng tạo thành phần thân của hàm được đưa vào từ đây.

3.1.1 NHỮNG HÀM CÓ SẴN TRONG PYTHON

Có nhiều hàm có sẵn trong python đã được giới thiệu trước đó như là type, len, … Ngoài ra, chúng tôi đã giới thiệu các hàm khác nhau có sẵn từ các gói khác nhau, ví dụ: math.sin, np.array, v.v. Bạn vẫn còn nhớ cách gọi và sử dụng các hàm này chứ?

TRY IT! Chứng minh hàm **len** là hàm có sẵn bằng cách

dùng hàm **type.**

In [1]: type(len)

Out[1]: builtin_function_or_method

TRY it! Chứng minh **np.linspace** là một hàm bằng cách dùng hàm **type**. Sau đó, Tìm ra cách sử dụng hàm bằng dấu chấm hỏi.

```
In [2]: import numpy as np
        type(np.linspace)
Out[2]: function
```

3.1.2 TẠO HÀM

Chúng ta có thể tự tạo nên một hàm của riêng mình. Một hàm có thể được tạo theo nhiều cách. Cách phổ biến nhất để định nghĩa một hàm là gọi nó bằng cách sử dụng từ khóa def, như hình dưới đây:

```
def function_name(parameter_1, parameter_2, ...):
    """
    Descriptive String
    """
    # comments about the statements
    function_statements

    return output_parameters (optional)
```

Việc xác định một hàm Python yêu cầu hai thành phần sau:

1. **Tiêu đề** hàm bắt đầu bằng một def từ khóa, tiếp theo là một cặp dấu ngoặc đơn với các tham số đầu vào bên trong và kết thúc bằng dấu hai chấm (:);

2. **Thân hàm** là một khối thụt vào (thường là bốn khoảng trắng) cho biết phần thân chính của chứ hàm. Nó bao gồm ba phần:

• Chuỗi mô tả, là chuỗi mô tả hàm có thể được truy cập bằng hàm help() hoặc dấu chấm hỏi. Dấu ngoặc kép ba hoặc ba dấu ngoặc kép cho biết nơi đặt (hoặc định vị) các chuỗi mô tả của bạn. Bạn có thể viết bất kỳ chuỗi nào bên trong dấu ngoặc kép, trong một dòng hoặc nhiều dòng.

• Các câu lệnh hàm, là các hướng dẫn từng bước mà hàm sẽ thực thi khi gọi hàm. Lưu ý rằng có một dòng bắt đầu bằng #; Đây là một nhận xét dòng duy nhất, có nghĩa là nó không phải là một phần của hàm và không thể được thực thi.

• Các câu lệnh trả về, có thể chứa một số tham số được trả về sau khi hàm được gọi và sẽ được giải thích kĩ hơn ở phần sau. Bất kỳ loại dữ liệu nào cũng có thể được trả về, thậm chí là một hàm.

NOTE! Khi nhập một tham số hay đối số. Trong đó tham số là một biến được xác định bởi một hàm nhận được một giá trị khi hàm được gọi. Đối số là một giá trị được truyền đến một phương thức khi nó được gọi. Ví dụ: nếu chúng ta định nghĩa một hàm hello(name), thì name là một tham số đầu vào. Khi chúng ta gọi hàm và truyền vào một giá trị 'Qingkai', thì giá trị này là một đối số đầu vào. Do sự khác biệt rất tinh tế, trong phần còn lại của cuốn sách, chúng ta sẽ sử dụng các tham số và đối số thay thế cho nhau.

TIP! Khi mã code của bạn trở nên dài hơn và phức tạp hơn, nhận xét có thể giúp bạn và những người đọc mã code của bạn điều hướng qua các lệnh và cung cấp một "lộ trình" hợp lý để hiểu những gì bạn đang cố gắng làm. Tập thói quen bình luận thường xuyên sẽ ngăn ngừa lỗi coding, hiểu mã code của bạn sẽ đi về đâu khi bạn viết và hỗ trợ bạn tìm lỗi khi mắc lỗi. Mặc dù nó là tùy chọn, người ta cũng thường đặt mô tả về hàm, tác giả và ngày tạo trong chuỗi mô tả dưới tiêu đề hàm (bạn có thể bỏ qua chuỗi mô tả). Chúng tôi thực sự khuyên bạn nên nhận xét nhiều bằng mã của riêng bạn.

TRY IT! Xác định một hàm có tên my_adder có ba số và tính tổng của chúng

```
In [4]: def my_adder(a, b, c):
            """
            function to sum the 3 numbers
            Input: 3 numbers a, b, c Output:
            the sum of a, b, and c author:
            date:
            """

            # this is the summation
            out = a + b + c

            return out
```

WARNING! Nếu bạn không thụt lề mã của mình khi xác định một hàm, máy sẽ báo là **IndentationError**

```
In [5]: def my_adder(a, b, c):
        """
        function to sum the 3 numbers
        Input: 3 numbers a, b, c
        Output: the sum of a, b, and
        c author:
        date:
        """

        # this is the
        summation
        File "<ipython-input-5-e6a61721f00e>", line 8
        """

    ^
    IndentationError: expected an indented block
```

TIP! Nhập thủ công bốn khoảng trắng là một mức thụt lề. Các mức thụt lề sâu hơn được yêu cầu khi bạn có các hàm lồng nhau hoặc if-statements, mà chúng ta sẽ thảo luận trong chương tiếp theo. Lưu ý rằng đôi khi bạn cần thụt lề hoặc bỏ thụt lề một khối mã. Bạn có thể thực hiện việc này bằng cách trước tiên chọn tất cả các dòng trong khối mã, sau đó nhấn Tab và Shift + Tab để tăng hoặc giảm một mức thụt lề.

TIP! Xây dựng các phương pháp mã hóa tốt bằng cách đưa ra các biến và chức năng tên mô tả, nhận xét thường xuyên và tránh các dòng mã không liên quan.

Ngược lại, hãy xem xét chức năng sau đây thực hiện nhiệm vụ tương tự như my_adder nhưng không được xây dựng bằng các phương pháp hay nhất. Như bạn có thể thấy, việc tuân theo logic của mã và ý định của tác giả là vô cùng khó khăn.

VÍ DỤ: Mặt kém hiệu quả của my_adder.

```
In [6]: def abc(a, s2, d):
        z = a + s2
        z = z + d
        x = z
        return x
```

Các hàm phải phù hợp với sơ đồ đặt tên tương tự như các biến. Chúng chỉ có thể chứa các ký tự alphanu- meric và dấu gạch dưới, và ký tự đầu tiên phải là một chữ cái.

TIP! Như quy ước với tên biến, tên hàm phải là chữ thường, với các từ được phân tách bằng dấu gạch dưới khi cần thiết để cải thiện khả năng đọc.

TRY IT! Sử dụng my_adder hàm của bạn để tính tổng của một vài số. Xác minh rằng kết quả là chính xác. Hãy thử gọi hàm trợ giúp trên my_adder.

```
In [7]: d = my_adder(1, 2, 3)
        d

Out[7]: 6

In [8]: d = my_adder(4, 5, 6)
        d

Out[8]: 15

In [9]: help(my_adder)

Help on function my_adder in module

main_: my_adder(a, b, c)
   function to sum the 3 numbers
   Input: 3 numbers a, b, c
   Output: the sum of a, b, and c
   author:
   date:
```

WHAT IS HAPPENING? Đầu tiên hãy nhớ lại rằng toán tử phân công hoạt động từ phải sang trái. Điều này có nghĩa là my_adder (1,2,3) được giải quyết trước khi chuyển nhượng

cho d.

1. Python tìm thấy hàm my_adder.
2. my_adder lấy giá trị đối số đầu vào đầu tiên 1 và gán nó cho biến có tên a (tên biến đầu tiên trong danh sách đối số đầu vào).
3. my_adder lấy giá trị đối số đầu vào thứ hai 2 và gán nó cho biến có tên B (tên biến thứ hai trong danh sách đối số đầu vào).
4. my_adder lấy giá trị đối số đầu vào thứ ba 3 và gán nó cho biến có tên C (tên biến thứ ba trong danh sách đối số đầu vào).
5. my_adder tính tổng của a, b và c, là 1 + 2 + 3 = 6.

6. my_adder gán giá trị 6 cho biến ra.

7. my_adder xuất ra giá trị chứa trong biến đầu ra, là 6

8. my_adder(1,2,3) tương đương với giá trị 6 và giá trị này được gán cho biến có tên d.

Python cho phép người dùng tự do rất nhiều để gán các biến cho các loại dữ liệu khác nhau. Ví dụ: có thể cung cấp cho biến x một từ điển hoặc một giá trị nổi. Trong các ngôn ngữ lập trình khác, điều này không phải lúc nào cũng đúng. Trong các chương trình này, bạn phải khai báo vào đầu phiên cho dù x sẽ là từ điển hay kiểu nổi và một khi bạn quyết định loại đó là loại nào, bạn không thể thay đổi nó. Đây có thể là cả một lợi ích và một nhược điểm. Ví dụ, my_adder được xây dựng với giả định rằng các đối số đầu vào là các loại số, int hoặc float; tuy nhiên, người dùng có thể vô tình nhập một danh sách hoặc chuỗi vào my_adder, điều này không chính xác. Nếu bạn cố gắng nhập một đối số đầu vào kiểu phi số vào my_adder, Python sẽ tiếp tục thực thi hàm cho đến

TRY IT! Use the string "1" as one of the input arguments to `my_adder`; in addition, use a list as one of the input arguments to `my_adder`.

```
In [10]: d = my_adder("1", 2, 3)
```

```
TypeError                Traceback (most recent call last)

<ipython-input-10-245d0f4254a9> in <module>
----> 1 d = my_adder("1", 2, 3)

<ipython-input-4-72d064c3ba7a> in my_adder(a, b, c)
      9
     10    # this is the summation
---> 11    out = a + b + c
     12
     13    return out

TypeError: must be str, not int
```

```
In [11]: d = my_adder(1, 2, [2, 3])
```

```
TypeError                Traceback (most recent call last)
```

khi có sự cố.

```
    <ipython-input-11-04f0428ffc51> in <module>
----> 1 d = my_adder(1, 2, [2, 3])


    <ipython-input-4-72d064c3ba7a> in my_adder(a, b, c)
      9
     10    # this is the summation
---> 11    out = a + b + c
     12
     13    return out


    TypeError: unsupported operand type(s) for +: "int"
    and "list"
```

TIP! Hãy nhớ đọc các thông báo lỗi mà Python cung cấp. Họ thường cho bạn biết chính xác vấn đề ở đâu. Trong trường hợp này, lỗi cho biết ---> 11 out = a + b + c, có nghĩa là có lỗi trong my_adder trên dòng thứ 11. Lý do có lỗi là **TypeError**, vì (các) loại toán hạng không được hỗ trợ cho +: "int" và "list", có nghĩa là chúng tôi không thể thêm int và danh sách.

Tại thời điểm này, bạn không có bất kỳ quyền kiểm soát nào đối với những gì người dùng gán chức năng của bạn làm ar-guments đầu vào và liệu chúng có tương ứng với những gì bạn dự định cho các đối số đầu vào đó hay không. Vì vậy, hiện tại, hãy viết các hàm của bạn với giả định rằng chúng sẽ được sử dụng chính xác.

Bạn có thể giúp bản thân và những người dùng khác sử dụng chức năng của mình một cách chính xác bằng cách cung cấp nhận xét chi tiết về mã của bạn. Bạn có thể soạn các hàm bằng cách gán các lệnh gọi hàm làm đầu vào cho các hàm khác. Theo thứ tự các hoạt động, Python sẽ thực thi lệnh gọi hàm trong cùng trước. Bạn cũng có thể gán các biểu thức toán học làm đầu vào cho các hàm. Trong trường hợp này, Python sẽ thực thi các biểu thức toán học trước.

TRY IT! Sử dụng hàm my_adder để tính tổng của tội lỗi (π), cos (π) và tan (π). Sử dụng các biểu thức toán học làm đầu vào để my_adder và xác minh rằng hàm thực hiện các phép toán một cách chính xác.

```
In [12]: d=my_adder(np.sin(np.pi), np.cos(np.pi),
         np.tan(np.pi))
            d

Out[12]: -1.0

In [13]: d = my_adder(5 + 2, 3 * 4, 12/
         6) d

Out[13]: 21.0

In [14]: d = (5 + 2) + 3 * 4 + 12 / 6
            d

Out[14]: 21.0
```

Các hàm Python có thể có nhiều tham số đầu ra. Khi gọi một hàm có nhiều tham số đầu ra, bạn có thể giải nén kết quả bằng nhiều biến mà bạn nên phân tách bằng dấu phẩy. Về cơ bản, hàm này sẽ trả về nhiều tham số kết quả trong một bộ, sau đó cho phép bạn giải nén bộ trả về. Xem ví dụ sau (lưu ý rằng nó có nhiều tham số đầu ra):

VÍ DỤ: Tính toán hàm my_trig_sum cho a=2 và b=3. Gán param- eter đầu ra đầu tiên cho biến c, tham số đầu ra thứ hai cho biến d và tham số thứ ba cho biến e..

```
In [15]: def my_trig_sum(a, b):
             """
             function to demo return
             multiple author
             date """
             out1 = np.sin(a) + np.cos(b)
             out2 = np.sin(b) + np.cos(a)
             return out1, out2, [out1,
             out2]

In [16]: c, d, e = my_trig_sum(2, 3)
         print(f"c ={c}, d={d}, e={e}")

c =-0.0806950697747637, d=-0.2750268284872752,
e=[-0.0806950697747637, -0.2750268284872752]
```

Nếu bạn gán kết quả cho một biến, bạn sẽ nhận được một bộ bao gồm tất cả các tham số đầu ra.

THỬ! Tính toán hàm my_trig_sum cho a=2 và b=3. Xác minh đầu ra là một bộ.

```
In [17]: c = my_trig_sum(2, 3)
         print(f"c={c}, and the returned type is {type(c)}")

c=(-0.0806950697747637, -0.2750268284872752,
[-0.0806950697747637, -0.2750268284872752]),
and the returned type is <class "tuple">
```

Một hàm có thể được xác định mà không cần đối số đầu vào và trả về bất kỳ giá trị nào.

VÍ DỤ: Hàm không có chức năng nhập và xuất.

```
In [18]: def print_hello():
         print("Hello")

In [19]: print_hello()

Hello
```

NOTE! Thậm chí không có đối số đầu vào, khi bạn gọi hàm, bạn vẫn cần bao gồm các dấu ngoặc đơn.

Đối với đầu vào của đối số, chúng ta cũng có thể bao gồm giá trị mặc định. Xem ví dụ sau:

VÍ DỤ: Chạy hàm sau có và không có đầu vào:

```
In [20]: def print_greeting(day = "Monday", name =
         "Qingkai"): print(f"Greetings, {name}, today
         is {day}")

In [21]: print_greeting()

Greetings, Qingkai, today is

Monday


In [22]: print_greeting(name = "Timmy", day =

"Friday") Greetings, Timmy, today is Friday


In [23]: print_greeting(name = "Alex")
```

Chúng ta có thể thấy rằng nếu chúng ta gán một giá trị cho đối số khi chúng ta xác định hàm, giá trị này sẽ là giá trị mặc định của hàm. Nếu người dùng không cung cấp đầu vào cho đối số này, thì giá trị mặc định này sẽ được sử dụng trong quá trình gọi hàm. Lưu ý rằng thứ tự của đối số không quan trọng khi gọi hàm nếu bạn cung cấp tên của đối số.

3.2 Biến địa phương và biến toàn cục

Ở chương 2 đã giới thiệu ý tưởng về khối bộ nhớ liên kết với sổ lưu ý, nơi biến được tạo ra trong sổ lưu ý đã được lưu trữ. Một hàm luôn có khối bộ nhớ riêng dành cho các biến được tạo trong hàm đó. Khối bộ nhớ này không được chia sẻ với toàn bộ bộ nhớ trong khối bộ nhớ trong sổ lưu ý. Do đó, biến có tên có thể được gán trong một hàm mà không làm thay đổi với một biến khác có cùng tên ở hàm khác. Khối bộ nhớ được liên kết với hàm được mở mỗi khi hàm được sử dụng.

Thử dòng lệnh! Giá trị ở *out* sẽ là bao nhiêu sau khi các dòng lệnh sau được thực thiện? Lưu ý rằng nó không phải là giá trị *6*, đó là giá trị ra đã được chỉ định bên trong *my_adder*.

```
In [1]: def my_adder(a, b, c):
            out = a + b + c
            print(f"The value out within the function is {out}")
            return out
        out = 1
        d = my_adder(1, 2, 3)
        print(f"The value out outside the function is {out}")

        The value out within the function is 6
        The value out outside the function is 1
```

Trong *my_adder*, giá trị *out* là một **giá trị địa phương.** Đó là bởi vì nó được định nghĩa trong hàm *my_adder*, nó không ảnh hưởng gì đến giá trị bên ngoài của hàm. Các hành động được thực hiện bên trong sổ lưu ý bên ngoài hàm không thể ảnh hưởng đến nó, ngay cả khi có trùng tên.Vì vậy trong các ví dụ trước đây, có biến out, đã được xác định trong ô sổ lưu ý. Khi *my_adder* được gọi trong dòng tiếp theo, Python mở một khối bộ nhớ mới cho các biến của hàm đó. Một trong cách biến được tạo ra trong hàm là một biến hoàn toàn khác, out. Bởi vì chúng nằm trong một bộ nhớ khác, nên việc gán *out* trong *my_adder* không thay đổi giá trị gán của *out* ở ngoài hàm.

Tại sao có các khối bộ nhớ của hàm riêng biệt hơn là một khối bộ nhớ duy nhất? Mặc dù nó có vẻ không hợp lý cho Python tách khối bộ nhớ, nó rất hiệu quả cho các dự án lớn bao gồm nhiều hàm hoạt động cùng nhau. Nếu một lập trình viên chịu trách nhiệm viết lệnh một hàm và một hàm khác chịu trách nhiệm chạy lệnh một hàm khác, có các khối bộ nhớ riêng biệt cho phép mỗi lập trình viên làm việc độc lập và tự tin rằng lệnh của họ sẽ không tạo ra lỗi khi đang xem xét dòng lệnh của lập trình viên khác và ngược lại. Các khối bộ nhớ riêng biệt bảo vệ một hàm từ những ảnh hưởng bên ngoài. Những thứ duy nhất từ bên ngoài khối bộ nhớ của hàm có thể ảnh hưởng đến những gì xảy ra bên trong một hàm là các đối số đầu vào và những thứ duy nhất có thể thoát ra thế giới bên ngoài từ khối bộ nhớ của hàm khi hàm kết thúc là các đối số đầu ra.

Ví dụ: Hãy xem xét hàm sau.

```
In [2]: def my_test(a, b):
            x = a + b
            y = x * b
            z = a + b
            m = 2
            print(f"Within function: x={x}, y={y}, z={z}")
            return x, y
```

THỬ! Giá trị của a, b, x, y và z sẽ là bao nhiêu sau khi dòng lệnh sau được chạy?

```
In [3]: a = 2
        b = 3
        z = 1
        y, x = my_test(b, a)
        print(f"Outside function: x={x}, y={y}, z={z}")

        Within function: x=5, y=10, z=5
        Outside function: x=10, y=5, z=1
```

THỬ! Giá trị của a, b, x, y và z sẽ là bao nhiêu sau khi dòng lệnh sau được chạy?

```
In [4]: x = 5
        y = 3
        b, a = my_test(x, y)
        print(f"Outside function: x={x}, y={y}, z={z}")

        Within function: x=8, y=24, z=8
        Outside function: x=5, y=3, z=1
```

THỬ ! Giá trị của m sẽ là bao nhiêu nếu bạn in m ra bên ngoài hàm?

```
In [5]: m

        ---------------------------------------------------------------------------
        NameError                                 Traceback (most recent call last)
        ~\AppData\Local\Temp\ipykernel_4848\1497821502.py in <module>
        ----> 1 m

        NameError: name 'm' is not defined
```

Lưu ý rằng giá trị của m sẽ không được xác định bên ngoài hàm bởi vì nó đã được xác định bên trong hàm. Điều này cũng đúng nếu bạn xác định cho một biến bên ngoài hàm, việc sử dụng nó bên trong hàm sẽ làm thay đổi giá trị, và thông báo lỗi tương tự sẽ xảy ra.

Ví dụ: Cố gắng sử dụng và thay đổi giá trị n trong hàm.

Giải pháp là sử dụng từ khóa toàn cục để cho Python biết biến này là một biến toàn cục và có thể được sử dụng cả bên ngoài và bên trong hàm.

Ví dụ: Xác định n là biến toàn cục, sau đó sử dụng và thay đổi giá trị n trong hàm số.

```
In [7]: n = 42

        def func():
            global n
            print(f"Within function: n is {n}")
            n = 3
            print(f"Within function: change n to {n}")

        func()
        print(f"Outside function: Value of n is {n}")

        Within function: n is 42
        Within function: change n to 3
        Outside function: Value of n is 3
```

3.3 Hàm lồng

Khi bạn tao và lưu trữ một hàm mới, nó sẽ hoạt động giống như bất kỳ hàm tích hợp sẵn nào khác trong Python. Bạn có thể gọi hàm đó từ bất kỳ nơi nào trong sổ lưu ý, và bất kì hàm nào khác cũng có thể gọi lại trong hàm. **Hàm lồng** là hàm mà nó được xác định trong một hàm nào khác – **hàm chính**. Chỉ duy nhất hàm chính mới có thể gọi là hàm lồng. Nhớ rằng hàm lồng vẫn giữ lại một khối bộ nhớ riêng biệt với hàm chính của nó.

THỬ ! Hãy xem xét hàm sau và hàm lồng nhau:

```
In [ ]: import numpy as np
        def my_dist_xyz(x, y, z):
            """
            x, y, z là tọa độ 2D được chứa trong một bộ đầu ra:
            d - danh sách, khi
            d [0] là khoảng cách giữa x và y
            d [1] là khoảng cách giữa x và z
            d [2] là khoảng cách giữa y và z
            """

        def my_dist(x, y):
            """"
            hàm phụ của my_dist_xyz
            Đầu ra là khoảng cách giữa x và y,
            được tính bằng cách sử dụng công thức khoảng cách
            """
            out = np.sqrt((x[0]-y[0])**2+(x[1]-y[1])**2)
            return out
        d0 = my_dist(x, y)
        d1 = my_dist(x, z)
        d2 = my_dist(y, z)
        return [d0, d1, d2]
```

Lưu ý rằng giá trị của *x* và *y* xuất hiện trong cả *my_dist_xyz* và *my_dist*. Điều này được phép vì một hàm lồng nhâu có một khối bộ nhớ riêng biệt với hàm chính của nó. Các hàm lồng nhau rất là hữu ích khi một nhiệm vụ phải được thực hiện nhiều lần trong hàm nhưng không được thực hiện bên ngoài hàm. Trong cách này, các hàm lồng nhau giúp hàm chính thực hiện nhiệm vụ của nó trong khi ẩn trong hàm chính.

THỬ ! Gọi hàm *my_dist_xyz* cho *x = (0, 0), y = (0, 1), z = (1, 1)*. Cố gắng gọi hàm *my_dist* lồng nhau trong ô sau:

```
In [2]: d = my_dist_xyz((0, 0), (0, 1), (1, 1))
        print(d)
        d = my_dist((0, 0), (0, 1))

        [1.0, 1.4142135623730951, 1.0]

        ---------------------------------------------------------------------------
        NameError                                 Traceback (most recent call last)
        ~\AppData\Local\Temp\ipykernel_8288\3509818064.py in <module>
              1 d = my_dist_xyz((0, 0), (0, 1), (1, 1))
              2 print(d)
        ----> 3 d = my_dist((0, 0), (0, 1))

        NameError: name 'my_dist' is not defined
```

Ví dụ sau là dòng lện được lặp lài mà không sử dụng được hàm lồng nhau. Để ý rằng xem bao nhiêu sự bận rộn và hàm trông lộn xộn hơn và sự khó hiểu của hàm về những gì đang xảy ra. Phiên bản này dễ bị sai nhiều hơn vì bạn có ba cơ hội gõ nhầm công thức khoảng cách. Lưu ý rằng hàm này có thể được viết nhỏ gọn hơn bằng cách sử dụng các phép toán vectơ. Chúng tôi để lại một bài tập để ví dụ.

```
In [4]: import numpy as np
        def my_dist_xyz(x, y, z):
            """
            x, y, z are 2D coordinates contained in a tuple
            output:
            d - list, where
            d[0] is the distance between x and y
            d[1] is the distance between x and z
            d[2] is the distance between y and z
            """
            d0 = np.sqrt((x[0]-y[0])**2+(x[1]-y[1])**2)
            d1 = np.sqrt((x[0]-z[0])**2+(x[1]-z[1])**2)
            d2 = np.sqrt((y[0]-z[0])**2+(y[1]-z[1])**2)
            return [d0, d1, d2]
```

3.4 HÀM LAMBDA

Đôi khi việc xác định một hàm theo cách thông thường không phải là tối ưu, đặc biệt nếu hàm của chúng ta chỉ là một dòng. Trong trường hợp này, chúng tôi sử dụng hàm ẩn danh trong Python, đây là một hàm được định nghĩa mà không có tên. Các loại hàm này còn được gọi là hàm lambda, vì chúng được định nghĩa bằng cách sử dụng từ khóa lambda. Một hàm lambda điển hình được định nghĩa như sau:

CONSTRUCTION:

```
lambda arguments: expression
```

Nó có thể có bất kỳ số đối số nào nhưng chỉ có một biểu thức.

TRY IT! Định nghĩa một hàm lambda bình phương số đầu vào; gọi hàm với đầu vào 2 và 5.

```
In [1]: square = lambda x: x**2
        print(square(2))
        print(square(5))
        4
        25
```

Trong hàm lambda ở trên, x là đối số và x ** 2 là biểu thức được đánh giá và trả về. Bản thân hàm không có tên và nó trả về một đối tượng hàm (được thảo luận trong chương sau) để bình phương nó. Sau khi nó được định nghĩa, chúng ta có thể gọi nó như một hàm bình thường. Hàm lambda tương đương với:

```
Def square (x) :
      return x**2
```

TRY IT! Xác định một hàm lambda, thêm x và y.

```
In [2]: my_adder = lambda x, y: x + y
          print(my_adder(2, 4))

6
```

Các hàm Lambda có thể hữu ích trong nhiều trường hợp và chúng tôi sẽ cung cấp các ví dụ khác trong các chương sau. Ở đây chúng tôi chỉ trình bày một trường hợp sử dụng phổ biến cho hàm lambda.

EXAMPLE: Sắp xếp [(1, 2), (2, 0), (4, 1)]dựa trên mục thứ hai trong bộ tuple..

```
In [3]: sorted([(1, 2), (2, 0), (4, 1)], key=lambda x: x[1])

Out[3]: [(2, 0), (4, 1), (1, 2)]
```

Điều gì xảy ra? Hàm được sắp xếp có một khóa đối số, trong đó một hàm khóa tùy chỉnh có thể được cung cấp để tùy chỉnh thứ tự sắp xếp. Chúng tôi sử dụng hàm lambda làm phím tắt cho hàm phím tùy chỉnh này.

3.5 HÀM DƯỚI DẠNG ĐỐI SỐ CHO HÀM

Cho đến nay, bạn đã gán các cấu trúc dữ liệu khác nhau cho các tên biến. Việc có thể gán cấu trúc dữ liệu cho một biến cho phép truyền thông tin đến các chức năng khác nhau và lấy lại thông tin từ chúng một cách gọn gàng và có trật tự. Đôi khi rất hữu ích khi có thể truyền một hàm và để nó hoạt động như một biến cho một hàm khác. Nói cách khác, đầu vào cho một số chức năng có thể là các chức năng khác. Trong phần trước, chúng ta đã thấy hàm lambda trả về một đối tượng hàm cho biến. Trong phần này, chúng ta sẽ thấy các ví dụ bổ sung về cách đối tượng hàm có thể được sử dụng làm đầu vào cho một hàm khác..

TRY IT! Gán hàm cực đại cho biến f. Xác minh loại `f`.

```
In [1]: f = max
      print(type(f))

<class "builtin_function_or_method">
```

Trong ví dụ trước, f bây giờ tương đương với hàm max. Bởi vì x = 1 có nghĩa là x và 1 có thể hoán đổi cho nhau, hàm f và max hiện có thể hoán đổi cho nhau.

TRY IT! Tìm giá trị tối đa từ danh sách [2, 3, 5] bằng cách sử dụng f. Xác minh rằng kết quả giống như sử dụng tối đa.

```
In [2]: print(f([2, 3, 5]))
        print(max([2, 3, 5]))

5
```

TRY IT! Viết một hàm my_fun_plus_one nhận một đối tượng hàm, f và một số thực x làm đối số đầu vào; my_fun_plus_one sẽ trả về f được đánh giá tại x và kết quả được thêm vào giá trị 1. Xác minh rằng nó hoạt động với các hàm và giá trị khác nhau của x.

```
In [3]: import numpy as np

        def my_fun_plus_one(f, x):
            return f(x) + 1

        print(my_fun_plus_one(np.
        sin, np.pi/2))
        print(my_fun_plus_one(np.
        cos, np.pi/2))
        print(my_fun_plus_one(np.
        sqrt, 25))

2.0
1.0
6.0
```

Trong ví dụ trên, các hàm khác nhau được sử dụng làm đầu vào cho hàm. Tất nhiên, chúng ta cũng có thể sử dụng các hàm lambda.

```
In [4]: print(my_fun_plus_one(lambda x: x + 2, 2))

5
```

3.6 TỔNG KẾT VÀ BÀI TẬP

3.6.1 TỔNG KẾT

1. Một hàm là một tập hợp các hướng dẫn khép kín được thiết kế để thực hiện một tác vụ cụ thể.
2. Một hàm có khối bộ nhớ riêng cho các biến của nó. Thông tin chỉ có thể được thêm vào khối bộ nhớ của hàm thông qua các biến đầu vào của hàm. Thông tin

chỉ có thể rời khỏi khối mem- ory của hàm thông qua các biến đầu ra của hàm.

3. Một hàm có thể được định nghĩa trong một hàm khác, được gọi là hàm lồng nhau. Hàm lồng nhau này chỉ có thể được truy cập bởi hàm cha.
4. Bạn có thể định nghĩa một hàm ẩn danh bằng cách sử dụng từ khóa lambda, được gọi là hàm lambda.
5. Bạn có thể gán hàm cho các biến bằng cách sử dụng bộ điều khiển hàm

3.6.2 BÀI TẬP

1. Viết hàm my_checker_board(n) xuất ra m trong đó m là mảng *nxn* array có dạng như sau:

$$m = \begin{matrix} 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 1 \end{matrix}$$

Lưu ý rằng phần tử phía trên bên trái phải luôn là 1. Giả sử rằng n là một số nguyên dương nghiêm ngặt.

```
In [27]: import numpy as np
         def my_checker_board(n):
             m = np.zeros((n,n),dtype=int)
             m[1::2, ::2]=1
             m[::2, 1::2]=1
             print(m)
         n = int(input('Nhap n: '))
         my_checker_board(n)

         Nhap n: 4
         [[0 1 0 1]
          [1 0 1 0]
          [0 1 0 1]
          [1 0 1 0]]
```

CHƯƠNG 4: CÂU LỆNH RẼ NHÁNH

4.1 CÂU LỆNH IF-ELSE

Câu lệnh rẽ nhánh, câu lệnh If-Else hay câu lệnh If là một cấu trúc mã chỉ thực thi cái khối mã nếu các điều kiện của nó được đáp ứng. Cấu điều kiện này được biểu diễn dưới dạng biểu thức logic.

Cấu trúc: Câu lện If-Else đơn giản

if logical expression:

code block 1

else:

code block 2

Chữ "if" ở đây đóng vai trò là một từ khóa. Khi Pythons nhận được một câu lệnh if, nó sẽ xác định xem biểu thức logic là đúng. Nếu câu lệnh là đúng, thì dòng lệnh trong cách khối sẽ được thực hiện. Nếu nó là sai, thì dòng lệnh trong câu lệnh if sẽ không được thực hiện. Cách để nhận biết điều này là "Nếu biểu thức logic là đúng thì dòng lệnh trong các khối sẽ được thực hiện". Tương tự, nếu biểu thức logic trong câu lệnh if-else là đúng thì dòng mã trong 'code block 1' sẽ được thực hiện. Ngược lại, 'code block 2' sẽ được thực hiện.

Khi có một số điều kiện cần xem xét, bạn có thể bao gồm các câu lệnh elif; nếu bạn muốn điều kiện bao gồm tất cả các trường hợp, thì bạn nên sử dụng câu lệnh khác. Để P, Q và R là ba biểu thức logic trong Python. Ví dụ sau cho thấy nhiều nhánh.

Thử câu lệnh! Viết hàm *my_thermo_stat(temp, desired_temp).* Trả về giá trị của hàm là một chuỗi string *"Heat"* nếu *temp* thấp hơn *desired_temp* trừ đi 5 độ, *"AC"* nếu *temp* cao hơn *desired_temp* cộng 5 độ, và ngược lại trả giá trị về là *"off"*.

```
Thay đổi trạng thái của bộ điều nhiệt dựa trên temperature and desired temperature
author
date
:type temp: Int
:type desiredTemp: Int
:rtype: String
```

```
In [1]: def my_thermo_stat(temp, desired_temp):
            if temp < desired_temp - 5:
                status = "Heat"
            elif temp > desired_temp + 5:
                status = "AC"
            else:
                status = "off"
            return status

In [2]: status = my_thermo_stat(65,75)
        print(status)

        Heat

In [3]: status = my_thermo_stat(75,65)
        print(status)

        AC

In [4]: status = my_thermo_stat(65,63)
        print(status)

        off
```

VÍ DỤ: Giá trị của y sẽ là bao nhiêu sau khi dòng lệnh sau được thực thi?

```
In [5]: x = 3
        if x > 1:
            y = 2
        elif x > 2:
            y = 4
        else:
            y = 0
        print(y)

        2
```

VÍ DỤ: Giá trị của y sẽ là bao nhiêu sau khi dòng lệnh sau được thực thi?

```
In [6]: x = 3
        if x > 1 and x < 2:
            y = 2
        elif x > 2 and x < 4:
            y = 4
        else:
            y = 0
        print(y)

        4
```

Lưu ý nếu bạn muốn câu lệnh logic $a < x < b$, đây được coi là hai câu lệnh điều kiện, $a < x$ và $x < b$. Python cũng cho phép bạn nhập $a < x < b$. Ví dụ,

```
In [7]: x = 3
        if 1 < x < 2:
            y = 2
        elif 2 < x < 4:
            y = 4
        else:
            y = 0
        print(y)

        4
```

Một câu lệnh được gọi là **lồng nhau** nếu nó hoàn toàn nằm trong một câu lệnh khác có cùng kiểu với nó. Ví dụ, **câu lệnh If lồng nhau** là câu lệnh if mà nó nằm hoàn toàn trong mệnh đề của một câu lệnh If khác

Ví dụ: Hãy suy nghĩ sẽ ra sao khi mà những dòng lệnh được thực hiên. Khả năng nào sẽ xuất hiện kết quả dựa trên các giá trị đầu vào là *x* và *y*?

"""

Ví dụ câu lệnh phân nhánh lồng nhau

author

date

:type x: Int

:type y: Int

:rtype: Int

"""

```
In [8]: if x > 2:
            if y < 2:
                out = x + y
            else:
                out = x - y
        else:
            if y > 2:
                out = x*y
            else:
                out = 0
        return out

        4
```

Lưu ý! Trước đó, Python sẽ cung cấp cùng một mức độ về sự thụt lùi của mỗi dòng lệnh trong câu điều kiện. Lệnh If lồng nhau nên được thụt vào bởi thêm bốn khoảng cách trắng. Bạn sẽ bị lỗi *IndentationError* nếu thụt vào không đúng khoảng cách, như cách chúng ta đã được nhìn thấy khi thảo luận về phần xác định hàm.

```
In [9]: import numpy as np

In [10]: all([1, 1, 0])
Out[10]: False
```

Có rất nhiều hàm logic được thiết kế để giúp bạn tạo nên câu lệnh phân nhánh. Ví dụ, bạn tự hỏi rằng nếu một biến có biến dữ liệu nhất định với hàm *isinstance*. Có nhiều những hàm khác có thể cho bạn biết những thông tin về những mảng logic chẳng hạn như *like*, sẽ tính đúng nếu bất kỳ phần tử nào trong mảng đúng, và ngược lại là sai, và *all*, sẽ chỉ tính đúng nếu tất cả các phần tử trong mảng đó đều đúng.

Đôi khi bạn muốn thiết kế một hàm cho riêng bạn để kiểm tra những đầu vào của hàm để chắc rằng những hàm của bạn được sử dụng đúng cách. Ví dụ, hàm *my_adder* ở chương trước mong đợi rằng giá trí tăng gấp đôi so với giá trị đầu vào. Nếu người sử dụng nhập vào *list* or a *string* như là một trong những biến đầu nhập vào, thì sau đó hàm sẽ gặp lỗi hoặc sẽ có những kết quả không mong muốn. Để tránh điều này, bạn cần đặt một cái kiểm tra để nói với người sử dụng hàm là nó sẽ không được sử dụng đúng cách. Điều đó và một vài những kĩ thuật khác để kiểm soát lỗi sẽ được mở rộng nhiều hơn ở Chapter 10. Cho đến bây giờ, bạn chỉ cần biết rằng chúng ta có thể sử dụng lệnh *raise* với *TypeError* để dừng thực thi một hàm và tạo ra một lỗi với một văn bản cụ thể.

Ví Dụ! Sửa đổi *my_adder* để đưa ra một lời cảnh cáo nếu người sử dụng không nhập đầu vào là một giá trị số. Hãy thử cho hàm đầu vào của bạn không phải là một giá trị số để kiểm tra thử nó hoạt động ra sao. Khi mà dòng lệnh quá dài, chúng ta có thể sử dụng kí hiệu "\" để tách hàng trong trường hợp nhiều hàng.

```
In [11]: def my_adder(a, b, c):
             """
             Calculate the sum of three numbers
             author
             date
             """
         # Check for erroneous input
             if not (isinstance(a, (int, float)) \
                 or isinstance(b, (int, float)) \
                 or isinstance(c, (int, float))):

                 raise TypeError("Inputs must be numbers.")
         # Return output
                 return a + b + c
```

```
In [12]: x = my_adder(1,2,3)
         print(x)n

         6
```

```
In [13]: x = my_adder("1","2","3")
         print(x)

         ---------------------------------------------------------------------------
         TypeError                                 Traceback (most recent call last)
         ~\AppData\Local\Temp\ipykernel_4024\2030836477.py in <module>
         ----> 1 x = my_adder("1","2","3")
               2 print(x)

         ~\AppData\Local\Temp\ipykernel_4024\2348971917.py in my_adder(a, b, c)
              10         or isinstance(c, (int, float))):
              11
         ---> 12         raise TypeError("Inputs must be numbers.")
              13 # Return output
              14         return a + b + c

         TypeError: Inputs must be numbers.
```

Có rất nhiều giá trị đầu vào sai mà hàm của bạn có thể bị phải từ người sử dụng, và không hợp lí lắm rằng hàm của bạn có thể chạy tốt với tất cả các lỗi đó. Do vậy, trừ khi có những điều ngược lại, hãy viết hàm của bạn giả sử chạy được với tất cả các trường hợp được sử dụng.

Phần còn lại của chương sẽ đưa ra một vài ví dụ các câu lệnh rẽ nhánh.

Thử câu lệnh! Viết một hàm gọi là *is_odd* sau đó gọi lại *"odd"* nếu giá trị đầu vào là giá trị số lẻ và "*even*" nếu giá trị là số chẵn. Bạn có thể giả sử đầu vào sẽ là một giá trị nguyên dương

```
In [14]: def is_odd(number):
             """
             hàm trả về "odd" nếu giá trị đầu vào là số lẻ,
             "even" ngược lại
             author
             date
             :type number: Int
             :rtype: String
             """
             # sử dụng modulo để kiểm tra nếu giá trị chia hết cho 2
             if number % 2 ==0:
                 # nếu chia hết cho 2, thì giá trị đó không lẻ
                 return "even"
             else:
                 return "odd"

In [15]: is_odd(11)
Out[15]: 'odd'

In [16]: is_odd(12)
Out[16]: 'even'
```

Thử câu lệnh! Viết một hàm gọi là *my_cric_calc* lấy r là một số lượng, và một chuỗi ký tự *calc* là đối số nhập đầu vào. Bạn có thể giả sử rằng *r* là giá trị dương, và *calc* là một chuỗi "*area*" *hay* "*circumference*". Hàm *my_cric_calc* nên tính diện tích của vòng tròn với bán rính *r*, nếu chuỗi *calc* là một "*area*" và chu vi của hình tròn với bán rính *r*, nếu *calc* là một "*circumference*"

```
In [17]: np.pi
Out[17]: 3.141592653589793

In [18]: def my_circ_calc(r, calc):
             """
             Tính các số đo của vòng trong
             author
             date
             :type r: Int or Float
             :type calc: String
             :rtype: Int or Float
             """
             if calc == "area":
                 return np.pi*r**2
             elif calc == "circumference":
                 return 2*np.pi*r

In [19]: my_circ_calc(2.5, "area")
Out[19]: 19.634954084936208

In [20]: my_circ_calc(3, "circumference")
Out[20]: 18.84955592153876
```

LƯU Ý! Hàm ở đây không giới hạn từng giá trị đầu vào nhưng có thể thực hiện được bằng sử dụng tốt mảng *NumPy* (tức là, cùng cách hoạt động sẽ áp dụng cho từng phần của mảng). Hãy xem ví dụ sau đây khi chúng ta tính chu vi cho bán kính là *[2, 3, 4]* bằng cách sử dụng mảng *NumPy*.

```
In [21]: my_circ_calc(np.array([2, 3, 4]), "circumference")
Out[21]: array([12.56637061, 18.84955592, 25.13274123])
```

4.2 PHÉP TOÁN BA NGÔI

Hầu hết các ngôn ngữ lập trình đều có **toán tử ba ngôi**, thường được gọi là biểu thức điều kiện, cung cấp cơ chế sử dụng một dòng lệnh để đánh giá biểu thức đầu tiên nếu điều kiện là đúng; nếu không, nó sẽ đánh giá điều kiện thứ hai. Để triển khai toán tử bậc ba trong Python, hãy sử dụng cấu trúc được trình bày bên dưới:

Cấu trúc: toán tử bậc ba trong Python

expression_if_true if condition else expression_if_false

Ví Dụ: Toán tử ba ngôi

```
In [1]: is_student = True
        person = "Student" if is_student else "Not Student"
        print(person)

        Student
```

```
In [2]: is_student = True
        if is_student:
            person = "Student"
        else:
            person = "Not Student"
        print(person)

        Student
```

Toán tử ba ngồi cung cấp một cách đơn giản cho phân nhánh và làm cho dòng lệnh của chúng ta được gọn hơn. Chương tiếp theo, chúng tôi giới thiệu vai trò của nó trong việc hiểu danh sách và sẽ chứng minh nó hữu ích.

4.3 TỔNG KẾT VÀ BÀI TẬP

4.3.1 TỔNG KẾT

1. Câu lệnh phân nhánh (if-else) cho phép hàm thực hiện cách hành động khác nhau trong cách hoàn cảnh khác nhau.
2. Phép toán ba ngôi cho phép dòng đơn trong câu lệnh phân nhánh.

4.3.2 BÀI TẬP

Cho Q(x) là phương trình bậc 2 Q(x) = $a^2 + bx + c$ với a,b,c bất kì. Tìm nghiệm r khi Q(x) = 0. 2 nghiệm của phương trình được tính theo công thức sau:

$$r = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

Nếu đó là 2 nghiệm thực (i.e., b2 > 4ac), 2 nghiệm ảo (i.e., b2 < 4ac), hoặc 1 nghiệm kép sẽ được tính r = $\frac{-b}{2a}$.

Viết hàm my_n_roots(a,b,c), trong đó a,b,c là hệ số của Q(x). Hàm nên trả về 2 giá trị bao gồm: n_roots and r. Và n_roots bằng 2 khi Q có 2 nghiệm thực, 1 khi Q có 1 nghiệm, −2 khi Q có 2 nghiệm ảo, và r là dãy chứa tập nghiệm của Q.

In [81]:
```
import math
import numpy as np
def my_n_roots(a,b,c):
    if a==0 :
        if b==0:
            print('Phuong trinh vo nghiem')
        else:
            r = -c/b
            n = 1
            return n, r
    else :
        delta=(b**2) - 4*a*c
        if delta==0:
            n = 1
            r = -b/2*a
            return n, r
        elif delta>0:
            n = 2
            r = np.empty(n)
            r[0] = round((-b-math.sqrt(delta))/(2*a),2)
            r[1] = round((-b+math.sqrt(delta))/(2*a),2)
            return n, r
        else:
            n = -2
            r = np.empty(abs(n),dtype=complex)
            r[0] = complex(round(-b/(2*a),4),-(math.sqrt(abs(delta))/(2*a)))
            r[1] = complex(round(-b/(2*a),4),(math.sqrt(abs(delta))/(2*a)))
            return n, r
a = int(input('Nhap a:'))
b = int(input('Nhap b:'))
c = int(input('Nhap c:'))
n_roots,r = my_n_roots(a,b,c)
print(f'n_roots = {n_roots}, r = {r}')
```

CHƯƠNG 5: VÒNG LẶP

5.1 VÒNG LẶP FOR

Định nghĩa: Vòng lặp for là một tập các lệnh được lặp đi lặp lại với mỗi giá trị tương ứng trong dãy. Vòng lặp for còn được coi là vòng lặp hữu hạn bởi nó xác điểm bắt đầu và điểm kết thúc của dãy.

Cú pháp tổng quát của vòng lặp for như sau:

for biến lặp in dãy

Khối lệnh

Vòng lặp for gán phần tử đầu tiên của dãy cho biến lặp. Sau đó, các lệnh trong khối lệnh được thực thi. Tiếp theo, biến lặp được gán bằng phần tử tiếp theo trong dãy, khối lệnh lại được thực thi. Việc này được lặp cho đến khi không còn phần tử nào trong dãy để gán.

THỬ! Tính tổng các số nguyên trong khoảng từ 1 đến 3?

```
In [1]: n = 0
        for i in range(1, 4):
            n = n + i

        print(n)
```

ĐIỀU GÌ ĐÃ XẢY RA?

0. Đầu tiên, hàm range (1, 4) tạo một dãy số nguyên có giá trị từ 1 đến 3. Coi lại định nghĩa hàm range để quen với cách nó hoạt động. Hàm range có dạng: range (giá trị bắt đầu, giá trị kết thúc, bước nhảy), có thể tùy chọn bước nhảy, nếu không giá trị mặc định là 1.
1. Biến n được gán giá trị bằng 0.
2. Biến i được gán giá trị bằng 1.
3. Biến n được gán giá trị bằng n + i (0 + 1 = 1).
4. Biến i được gán giá trị bằn 2.
5. Biến n được gán giá trị bằng n + i (1 + 2 = 3).
6. Biến i được gán giá trị bằng 3.
7. Biến n được gán giá trị bằng n + i (3 + 3 = 6).
8. Vì không còn giá trị nào trong dãy để gán, vòng lặp này dừng với giá trị n = 6.

Bên dưới là một số ví dụ để giúp bạn hiểu hơn về cách vòng lặp for hoạt động. Vài ví dụ khác về những dãy mà chúng ta có thể dùng để lặp bao gồm: các phần tử của một bộ, các ký tự trong một chuỗi ký tự và các loại dữ liệu tuần tự khác.

VÍ DỤ: In lần lượt các ký tự trong chuỗi "banana"

```
In [2]: for c in "bannana":
            print(c)

b
a
n
n
a
n
a
```

Cách khác: Bạn có thể sử dụng chỉ số để trích từng ký tự, cách ở trên vẫn ngắn gọn hơn. Nhắc lại, độ dài của chuỗi có thể được xác định bằng hàm len, chúng ta có thể bỏ qua giá trị bắt đầu và chỉ truyền vào hàm range giá trị kết thúc.

```
In [12]: s = "banana"
         for i in range(len(s)):
             print(s[i])

b
a
n
a
n
a
```

VÍ DỤ: Cho một danh sách a gồm các số nguyên, tính tổng các phần tử có trong a.

```
In [13]: s = 0
         a = [2, 3, 1, 3, 3]
         for i in a:
             s += i
         print(s)

         12
```

Python có hỗ trợ hàm sum dùng để giải quyết ví dụ trên một cách tiện lợi hơn. Khó hơn một chút, nếu yêu cầu là tính tổng các phần tử có chỉ số chẵn, bạn sẽ thay đổi đoạn code trên như thế nào?

```
In [15]: s = 0
         for i in range(0,len(a),2):
             s += a[i]
         print(s)

         6
```

LƯU Ý! Ta chọn bước nhảy là 2 trong hàm range để duyệt các phần tử có chỉ số chẵn trong danh sách a. Một lối viết tắt thường được sử dụng trong Python là dùng toán tử +=. Trong Python hay nhiều ngôn ngữ lập trình khác, câu lệnh i += 1 tương đương với i = i + 1, điều này cũng tương tự với các toán tử khác như: -=, /=, *=.

VÍ DỤ: Tạo một dictionary và dùng vòng lặp for để duyệt các khóa và giá trị tương ứng.

```
In [6]: dict_a = {"One":1, "Two":2, "Three":3}

        for key in dict_a.keys():
            print(key, dict_a[key])

One 1
Two 2
Three 3
```

Trong ví dụ trên, ta dùng phương thức keys để truy xuất các khóa. Tiếp đến, ta dùng khóa để truy cập đến giá trị tương ứng. Một cách khác, ta có thể sử dụng phương thức items của dictionary, phương thức này sẽ trả về một đối tượng chứa danh sách các cặp khóa và giá trị tương ứng theo từng bộ. Ta có thể gán những cặp giá trị đó vào 2 biến tương ứng (gán theo bộ), xem ví dụ sau:

```
In [7]: for key, value in dict_a.items():
            print(key, value)

One 1
Two 2
Three 3
```

Lưu ý rằng chúng ta có thể sử dụng 2 biến lặp khác nhau cùng một lúc. Có những trường hợp ta có thể lặp nhiều biến cùng một lúc. Ví dụ, nếu ta có hai danh sách với cùng số lượng phần tử và ta muốn duyệt chúng cùng một lúc, khi đó ta dùng hàm zip. Xem thử ví dụ bên dưới. Hàm này hợp các giá trị của hai biến lặp và trả về các bộ giá trị tương ứng. với phần tử thứ i của bộ chứa phần tử thứ i của từng biến lặp.

```
In [8]: a = ["One", "Two", "Three"]
        b = [1, 2, 3]

        for i, j in zip(a, b):
            print(i, j)

One 1
Two 2
Three 3
```

VÍ DỤ: Viết hàm have_digits và truyền vào hàm một chuỗi ký tự bất kỳ. Biến xuất ra out sẽ nhận giá trị là 1 nếu chuỗi chứa chữ số, ngược lại sẽ nhận giá trị là 0. Bạn có thể dùng phương thức isdigit của chuỗi để kiểm chuỗi có chứa chữ số.

```
In [5]: def have_digits(s):
            out = 0
            # Duyệt chuỗi ký tự
            for c in s:
                # Kiểm tra ký tự có phải là chữ số hay không
                if c.isdigit():
                    out = 1
                    break
            return out

In [6]: out = have_digits("only4you")
        print(out)

        1

In [8]: out = have_digits("only for you")
        print(out)

        0
```

Dòng đầu tiên trong hàm have_digits giả sử rằng chuỗi s không chứa ký tự là chữ số. (Kết quả xuất ra là 0 hoặc False).

Chú ý đến từ khóa mới break. Khi được thực thi, từ khóa break sẽ ngay lập tức kết thúc vòng lặp for chứa nó. Nghĩa là, nếu nó nằm trong một vòng lặp for bị lồng bởi một vòng for khác, nó sẽ dừng vòng lặp for nằm trong cùng. Như trong ví dụ trên, lệnh break được thực hiện nếu ta tìm thấy một chữ số trong chuỗi. Nếu không có câu lệnh

break, hàm vẫn sẽ chạy ra kết quả chính xác. Nhưng bởi vì nhiệm vụ của ta là chỉ kiểm tra xem có chữ số chứa trong chuỗi hay không, nên khi tìm được một chữ số, ta không cần phải tiếp tục kiểm tra. Tương tự như vậy, nếu một người nào đó được giao việc tìm kiếm như trên và với một chuỗi ký tự dài, người đó sẽ dừng việc kiểm tra ngay khi tìm được một chữ số. Lệnh break được sử dụng trong những tình huống kết quả được tìm thấy trong quá trình lặp và bạn muốn dừng vòng lặp ngay lúc đó. Một lệnh khác tác động đến vòng lặp ít hơn đó là lệnh continue, lệnh này sẽ bỏ qua và không thực thi những đoạn code ngay sau nó ở lần lặp hiện tại và nhảy đến bước lặp tiếp theo của vòng lặp. Xem ví dụ sau, ta sử dụng từ khóa continue để bỏ qua hàm print và không in giá trị 2:

```
In [12]: for i in range(5):

             if i == 2:
                 continue

             print(i)

0
1
3
4
```

VÍ DỤ: Viết hàm my_dist_2_points (xy_points, xy) với đối số truyền vào xy_points là một danh sách gồm các tọa độ x-y của một điểm từ không gian Euclidean, xy là một danh sách chứa một tọa độ x-y, và biến xuất ra d là một danh sách chứa các khoảng cách từ xy đến các điểm chứa trong từng dòng của xy_points.

```
In [18]: import math
         def my_dist_2_points(xy_points,xy):
             """
             Tạo ra một danh sách chứa các khoảng cách
             giữa điểm xy đến các điểm chứa trong từng dòng
             của danh sách xy_points
             """

             d =[]
             for xy_point in xy_points:
                 dist = math.sqrt((xy_point[0] - xy[0])**2 + (xy_point[1] - xy[1])**2)
                 d.append(dist)
             return d
```

```
In [19]: xy_points = [[3,2], [2,3], [2, 2]]
         xy = [1, 2]
         my_dist_2_points(xy_points,xy)

Out[19]: [2.0, 1.4142135623730951, 1.0]
```

Giống với lệnh if, vòng lặp for cũng có thể được lồng vào nhau.

VÍ DỤ: Cho một mảng 2 chiều x = [5 6;7 8]. Dùng vòng lặp for lồng nhau để tính tổng các phần tử của x.

```
In [15]: x = np.array([[5, 6], [7, 8]])
         n, m = x.shape
         s = 0
         for i in range(n):
             for j in range(m):
                 s += x[i, j]

         print(s)

26
```

ĐIỀU GÌ ĐÃ XẢY RA?

1. Biến s, dùng để tính tổng các phần tử, được gán bằng 0.
2. Vòng for ngoài cùng bắt đầu lặp với biến i được gán bằng 0.
3. Vòng for được lồng bên trong bắt đầu lặp với biến j được gán bằng 0.
4. Giá trị của biến s tăng lên một lượng x[i, j] = x[0,0] = 5. Vậy s = 5.
5. Vòng lặp bên trong gán j = 1.
6. Giá trị của biến s tăng lên một lượng x[i, j] = x[0,1] = 6. Vậy s =11.
7. Vòng lặp bên trong kết thúc.
8. Vòng lặp bên ngoài gán i = 1.
9. Vòng for bên trong bắt đầu lặp với biến j được gán bằng 0.
10. Giá trị của biến s tăng lên một lượng x[i, j] = x[1,0] = 7. Vậy s =18.

11. Vòng lặp bên trong gán j = 0.
12. Giá trị của biến s tăng lên một lượng x[i, j] = x[1,1] = 8. Vậy s =26.
13. Vòng lặp bên trong kết thúc.
14. Vòng lặp bên ngoài kết thúc với s = 26.

CẨN THẬN! Đừng thay đổi giá trị của biến của bên trong vòng lặp for. Mặc dù điều này có thể thực hiện được nhưng nó sẽ khiến code của bạn phức tạp hơn và có thể xảy ra lỗi.

5.2 VÒNG LẶP WHILE

ĐỊNH NGHĨA: Vòng lặp while hay còn được gọi là vòng lặp không xác định là một tập các lệnh được lặp đi lặp lặp khi biểu thức logic vẫn đúng. Cú pháp tổng quát của vòng lặp while như sau:

```
while <biểu thức logic>:
    # đoạn code sẽ được lặp cho đến khi biểu thức logic sai.
    Khối lệnh
```

Khi Python xử lý vòng lặp while, nó sẽ kiểm tra biểu thức logic trước. Nếu biểu thức logic đúng, khối lệnh sẽ được thực thi. Sau đó, chương trình quay lại kiểm tra biểu thức logic. Nếu biểu thức logic sai, vòng lặp while sẽ kết thúc.

THỬ! Xác định số lần 8 chia 2 cho tới khi kết quả nhỏ hơn 1.

```
In [1]: i = 0
        n = 8

        while n >= 1:
            n /= 2
            i += 1

        print(f"n = {n}, i = {i}")

n = 0.5, i = 4
```

ĐIỀU GÌ ĐÃ XẢY RA?

1. Đầu tiên biến i được gán bằng 0. (dùng để đếm số lần thực hiên phép chia n cho 2).
2. Biến n được gán bằng 8 là giá trị hiện tại chia cho 2.
3. Bắt đầu vòng lặp while.
4. Python xét biểu thức n>=1 tức 8>=1, điều này đúng và khối lệnh được thực thi.
5. n được gán bằng n/2 = 8/2 = 4.
6. i tăng lên 1.
7. Python xét biểu thức n>=1 tức 4>=1, điều này đúng và khối lệnh được thực thi.
8. n được gán bằng n/2 = 4/2 = 2.
9. i tăng lên 2.
10. Python xét biểu thức n>=1 tức 2>=1, điều này đúng và khối lệnh được thực thi.
11. n được gán bằng n/2 = 2/2 = 1.
12. i tăng lên 3.
13. Python xét biểu thức n>=1 tức 1>=1, điều này đúng và khối lệnh được thực thi.
14. n được gán bằng n/2 = ½ = 0.5.
15. i tăng lên 4.
16. Python xét biểu thức n>=1 tức 0.5>=1, điều này sai và vòng lặp while kết thúc với giá trị i = 4.

Có thể bạn sẽ thắc mắc, "Sẽ thế nào nếu biểu thức logic luôn luôn đúng và không đổi?"; một câu hỏi hay. Nếu biểu thức logic đúng và đoạn code bên trong vòng lặp while không làm thay đổi giá trị của biểu thức logic thì kết quả là vòng lặp này sẽ lặp vô hạn. Vòng lặp vô hạn chạy mãi không dừng, hoặc sẽ dừng khi máy tính của bạn bị hỏng hoặc cạn kiệt bộ nhớ.

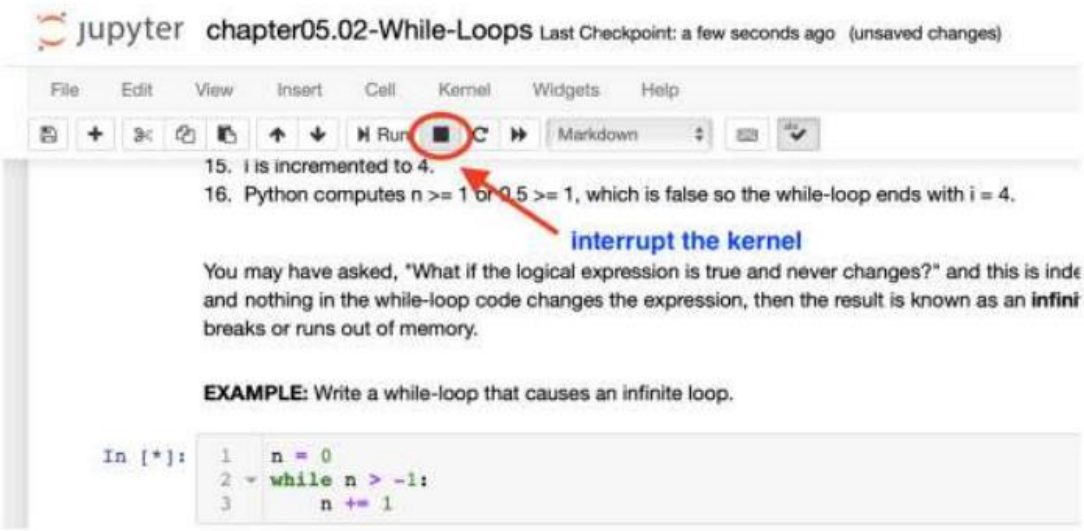
VÍ DỤ: Dùng vòng lặp while để tạo một vòng lặp vô hạn.

```
In [ ]: n = 0
        while n > -1:
            n += 1
```

Vì biến n sẽ luôn lớn hơn -1 dù cho vòng lặp có lặp bao nhiêu lần đi nữa, đoạn code này sẽ thực hiện mãi không dừng.

Bạn có thể dừng vòng lặp vô hạn bằng cách thủ công là nhấn nút interrupt the kernel (có hình vuông màu đen) ở trên thanh công cụ (xem hình bên dưới). Hoặc nút menu Kernel – Interrupt trong **notebook**. Hoặc nếu bạn sử dụng **Python shell**, bấm tổ hợp phím cmd + c trên Mac hoặc Ctrl + c trên Window.

Liệu có thể thay đổi 1 ký tự để vòng lặp while sẽ lặp ít nhất một lần nhưng không lặp vô hạn?



Đôi khi không dễ dàng để nhận ra các vòng lặp vô hạn. Xem thử hai ví dụ sau: một trong hai ví dụ này có chứa vòng lặp vô hạn. Bạn có thể phân biệt được đâu là vòng lặp vô hạn? Với những đoạn code phức tạp thì việc phát hiện vòng lặp vô hạn sẽ trở nên khó khăn.

VÍ DỤ: Vòng lặp while nào lặp vô hạn?

```
In [ ]: # Example 1
        n = 1
        while n > 0:
            n /= 2

In [ ]: # Example 2
        n = 2
        while n > 0:
            if n % 2 == 0:
                n += 1
            else:
                n -= 1
```

Đáp án: Ví dụ thứ nhất không phải là vòng lặp vô hạn bởi vì giá trị của biến n sẽ ngày càng nhỏ và Python không phân biệt được sự khác nhau của n và 0. Điều này sẽ được nói kỹ trong **CHƯƠNG 9**. Ví dụ thứ hai sẽ lặp vô hạn bởi vì n sẽ luôn thay đổi lần lượt giữa giá trị 2 và 3.

Giờ ta đã biết đến hai loại vòng lặp: vòng lặp for và vòng lặp while. Trong vài tình huống, cả hai vòng lặp có hiệu suất như nhau, nhưng đôi khi vòng lặp này cho hiệu suất tốt hơn khi giải quyết một số tác vụ nhất định so với cái còn lại. Thông thường, vòng lặp for được khuyên dùng khi số lần lặp được xác định trước, trong khi vòng lặp while được khuyên dùng khi số lần lặp là vô hạn hoặc không xác định được.

5.3 BAO HÀM

ĐỊNH NGHĨA: Trong Python, có nhiều cách khác nhau để duyệt vòng lặp; danh sách, dictionary, và tập **bao hàm** là những cách rất thông dụng. Một khi bạn làm quen với cách sử dụng chúng, bạn sẽ thấy chúng rất tiện và được sử dụng nhiều. **Bao hàm** cho phép tạo những dãy mới từ những dãy khác với cú pháp rất ngắn gọn. Đầu tiên, hãy nói về danh sách bao hàm.

5.3.1 Danh sách bao hàm

CẤU TRÚC: Danh sách bao hàm
[<giá trị xuất ra> <dãy truyền vào> <điều kiện>]

VÍ DỤ: Cho x = range (5), bình phương từng giá trị trong x và thêm vào danh sách y. Nếu ta không dùng danh sách bao hàm, đoạn code sẽ như sau:

```
In [1]: x = range(5)
        y = []

        for i in x:
            y.append(i**2)
        print(y)

[0, 1, 4, 9, 16]
```

Nếu dùng danh sách bao hàm, đoạn code sẽ gọn hơn.

```
In [2]: y = [i**2 for i in x]
        print(y)

[0, 1, 4, 9, 16]
```

Ta cũng có thể thêm điều kiện vào danh sách bao hàm. Ví dụ, nếu ta chỉ muốn thêm những số chẵn vào danh sách y ở ví dụ trên, ta chỉ cần thêm điều kiện vào danh sách bao hàm.

```
In [3]: y = [i**2 for i in x if i%2 == 0]
        print(y)

[0, 4, 16]
```

Nếu ta có hai vòng lặp lồng nhau, ta cũng có thể dùng danh sách bao hàm. Ví dụ, hai vòng lặp lồng nhau sau đây có thể thực bằng cách sử dụng danh sách bao hàm.

```
In [4]: y = []
        for i in range(5):
            for j in range(2):
                y.append(i + j)
        print(y)

[0, 1, 1, 2, 2, 3, 3, 4, 4, 5]

In [5]: y = [i + j for i in range(5) for j in range(2)]
        print(y)

[0, 1, 1, 2, 2, 3, 3, 4, 4, 5]
```

5.3.2 Thư viện bao hàm

Tương tự, ta cũng có thể sử dụng thư viện bao hàm. Xem ví dụ sau:

```
In [6]: x = {"a": 1, "b": 2, "c": 3}
        {key:v**3 for (key, v) in x.items()}

Out[6]: {"a": 1, "b": 8, "c": 27}
```

Ta có thể sử dụng tập bao hàm trong Python, nhưng phần này không được đề cập ở đây. Bạn sẽ phải tự tìm hiểu.

5.4 TỔNG KẾT VÀ CÁC BÀI TẬP

5.4.1 TÓM TẮT

1. Vòng lặp là một cơ chế dành cho các đoạn code phải thực hiện các tác vụ lặp đi lặp lại.
2. Có hai loại vòng lặp: vòng lặp for và vòng lặp while.
3. Vòng lặp rất quan trọng trong việc xây dựng các giải pháp lặp nhằm giải quyết các vấn đề.

5.4.2 BÀI TẬP

CÂU 5: Cho P, Q là hai ma trận. P là mảng có dạng m x p và Q có dạng p x n. M = P x Q được định nghĩa như sau:

M[i, j] = SIGMA P[i, k] x Q[k, j]. Viết hàm my_mat_mult(P, Q) dùng vòng lặp for để tính M. Gợi ý: bạn sẽ dùng ít nhất 3 vòng lặp for. Không được dùng hàm np.dot.

```
In [2]: import numpy as np
```

```
In [11]: def my_mat_mult(P,Q):

             """
             Hàm này dùng để nhân 2 ma trận:

             P có kích thước m x n
             Q có kích thước n x p
             Nếu hai ma trận không có kích thước phù hợp
             hàm sẽ trả về một câu thông báo.

             Đối số truyền vào: ma trận P, ma trận Q.
             Giá trị trả về: ma trận mới M có kích thước m x p.

             Người viết hàm: Distephano
             Ngày viết: 30/10/2022
             """
             # Dùng phương thức shape để lấy kích thước của 2 ma trận P, Q
             # Ma trận P có kích thước m x n
             m, n = np.shape(P)

             # Ma trận Q có kích thước a x b
             a, b = np.shape(Q)

             # Kiểm tra 2 ma trận P, Q có nhân được hay không, nếu không thì trả thông báo sau
             if a != n:
                 return 'Không thể nhân 2 ma trận này!'
```

```
             # M là kết quả của phép nhân 2 ma trận P, Q
             M = np.zeros((m,y))

             # Nhân hai ma trận P và Q
             for i in range(m):
                 for j in range(y):
                     # Tính các phần tử của ma trận mới
                     s = 0
                     for z in range (n):
                         s += P[i,z]*Q[z,j]
                     M[i, j] = s
```

```
In [15]: # Gọi hàm help để biết thông tin về hàm
         help(my_mat_mult)
```

```
Help on function my_mat_mult in module __main__:

my_mat_mult(P, Q)
    Hàm này dùng để nhân 2 ma trận:

    P có kích thước m x n
    Q có kích thước n x p
    Nếu không hai ma trận không có kích thước phù hợp
    hàm sẽ trả về một câu thông báo

    Đối số truyền vào: ma trận P, ma trận Q
    Giá trị trả về: ma trận mới M có kích thước m x p

    Người viết hàm: Distephano
    Ngày viết: 30/10/2022
```

Câu 11: Viết hàm my_n_fib_primes(n) với giá trị trả về là một danh sách chứa n số Fibonacci và đồng thời là số nguyên tố. Lưu ý 1 không phải số nguyên tố. Gợi ý: Không dùng đệ quy để tính dãy Fibonacci.

```
In [16]: # tạo ma trận P có các phần tử là 1
         P = np.ones((3,3))
         # Gọi hàm nhân 2 ma trận
         my_mat_mult(P,P)

Out[16]: array([[3., 3., 3.],
                [3., 3., 3.],
                [3., 3., 3.]])
```

```
In [18]: P = np.array([[1, 2, 3, 4], [5, 6, 7, 8]])
         Q = np.array([[1, 1, 1], [2, 2, 2], [3, 3, 3], [4, 4, 4]])
         # Gọi hàm nhân 2 ma trận
         my_mat_mult(P,Q)

Out[18]: array([[30., 30., 30.],
                [70., 70., 70.]])
```

```
In [11]: import math
```

```
In [19]: def my_n_fib_primes(n):
             """
             Hàm này dùng để tạo một danh sách
             gồm n số nguyên tố đầu tiên trong dãy Fibonacci

             Đối số truyền vào: số nguyên dương n.
             Giá trị trả về: Danh sách chứa n số nguyên tố đầu tiên trong dãy Fibonacci
             Nếu đối số truyền không hợp lệ, hàm sẽ trả về một câu thông báo.

             Người viết hàm: Distephano
             Ngày viết:: 30/10/2022
             """
             # Kiểm tra xem n có hợp lệ hay không
             if n < 1:
                 return 'n phải là số nguyên lớn hơn hoặc bằng 1.'

             # Khởi tạo d là một danh sách rỗng để chứa kết quả
             list_d = []

             # Khởi tạo hai giá trị đầu tiên của dãy fibonacci
             f1 = f2 = 1
```

```
    # Tìm n số fibonacci thỏa yêu cầu
    while n != 0:
        f = f1 + f2
        f1 = f2
        f2 = f
        # Biến check dùng để kiểm tra số nguyên tố, giá trị 1 mặc định là đúng
        check = 1
        # Kiểm tra xem số fibonacci vừa tính được có phải là số nguyên tố
        for j in range(2,int(math.sqrt(f)) + 1):
            if f % j == 0:
                check = 0
                break
        if check == 1:
            # Dùng phương thức append để thêm số fibonacci thỏa yêu cầu vào danh sách
            list_d.append(f)
            # Đếm số lần thêm vào danh sách d
            n -= 1
    return list_d
```

In [7]:
```
# Gọi hàm help để biết thông tin về hàm
help(my_n_fib_primes)
```

```
Help on function my_n_fib_primes in module __main__:

my_n_fib_primes(n)
    Hàm này dùng để tạo một danh sách
    gồm n số nguyên tố đầu tiên trong dãy Fibonacci

    Đối số truyền vào: số nguyên dương n.
    Giá trị trả về: Danh sách chứa n số nguyên tố đầu tiên trong dãy Fibonacci
    Nếu đối số truyền không hợp lệ, hàm sẽ trả về một câu thông báo.

    Người viết hàm: Distephano
    Ngày viết:: 30/10/2022
```

In [18]:
```
# Ví dụ khi đối số truyền vào không hợp lệ
my_n_fib_primes(0.5)
```

Out[18]:
```
'n phải là số nguyên lớn hơn hoặc bằng 1.'
```

In [16]:
```
# Với n = 8
my_n_fib_primes(8)
```

Out[16]:
```
[2, 3, 5, 13, 21, 55, 89, 233]
```

In [17]:
```
# Với n = 11
my_n_fib_primes(11)
```

Out[17]:
```
[2, 3, 5, 13, 21, 55, 89, 233, 377, 987, 1597]
```